

Machine Learning — Complete End-to-End Cheat Sheet

Coverage: 60% Theory · 40% Math, Examples & Diagrams

A complete reference from fundamentals to advanced algorithms.

Table of Contents

1. [ML Fundamentals](#)
2. [The ML Workflow](#)
3. [Key Math Foundations](#)
4. [Linear Regression](#)
5. [Logistic Regression](#)
6. [Decision Trees](#)
7. [Ensemble Methods](#)
8. [Support Vector Machines](#)
9. [K-Nearest Neighbors](#)
10. [Naive Bayes](#)
11. [Clustering \(Unsupervised\)](#)
12. [Dimensionality Reduction](#)
13. [Neural Networks & Deep Learning](#)
14. [Model Evaluation & Metrics](#)
15. [Regularization](#)
16. [Feature Engineering](#)
17. [Bias-Variance Tradeoff](#)
18. [Hyperparameter Tuning](#)
19. [Algorithm Comparison Table](#)
20. [Quick Reference Formulas](#)
21. [Transformers & Attention Mechanism](#)
22. [Reinforcement Learning](#)
23. [Anomaly Detection](#)
24. [Time Series & Forecasting](#)
25. [Natural Language Processing \(NLP\)](#)
26. [Generative Models \(GANs & VAEs\)](#)
27. [Model Interpretability \(XAI\)](#)
28. [MLOps & Production ML](#)

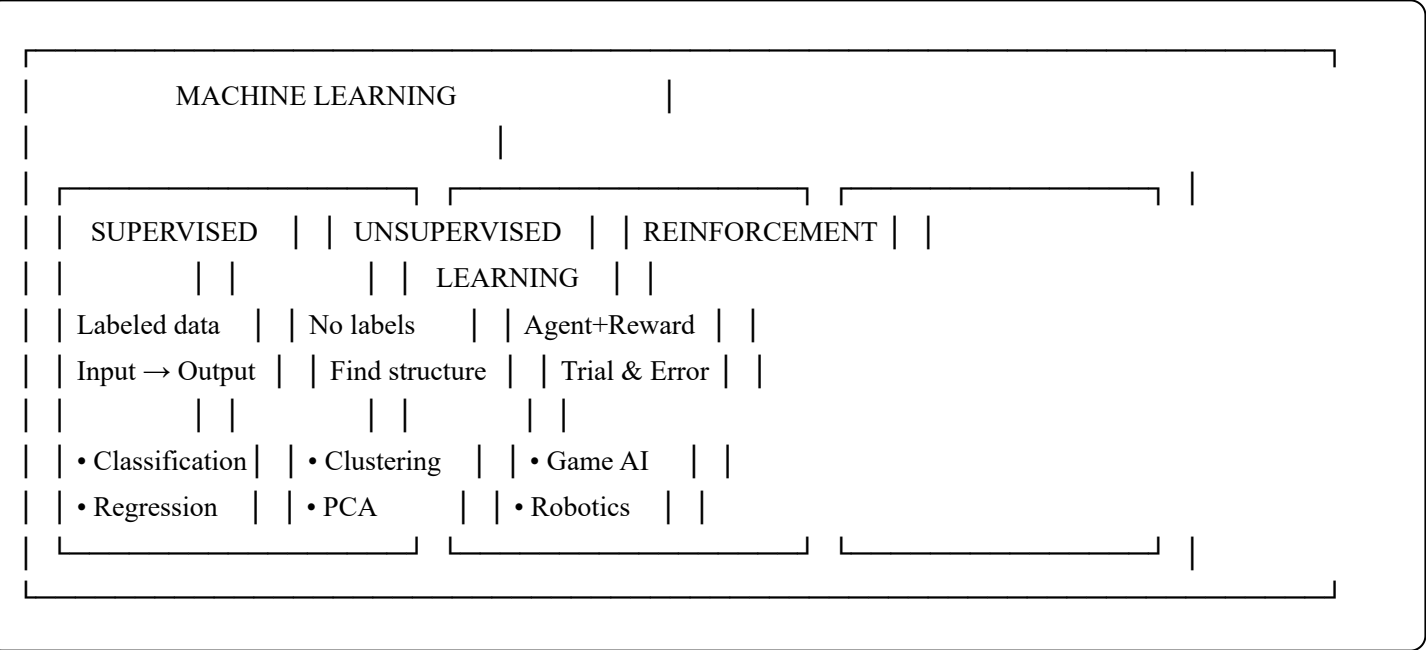
- 29. [Imbalanced Data Techniques](#)
- 30. [Advanced Optimization Algorithms](#)
- 31. [Probability Distributions in ML](#)
- 32. [Graph Neural Networks](#)
- 33. [Self-Supervised & Contrastive Learning](#)
- 34. [ML System Design Patterns](#)
- 35. [Python ML Ecosystem Cheatsheet](#)

1. ML Fundamentals

What is Machine Learning?

Machine Learning is a subset of Artificial Intelligence where systems **learn patterns from data** to make predictions or decisions, without being explicitly programmed for each task. The core idea: instead of writing rules manually, you feed examples to an algorithm, which discovers the rules itself.

Three Paradigms

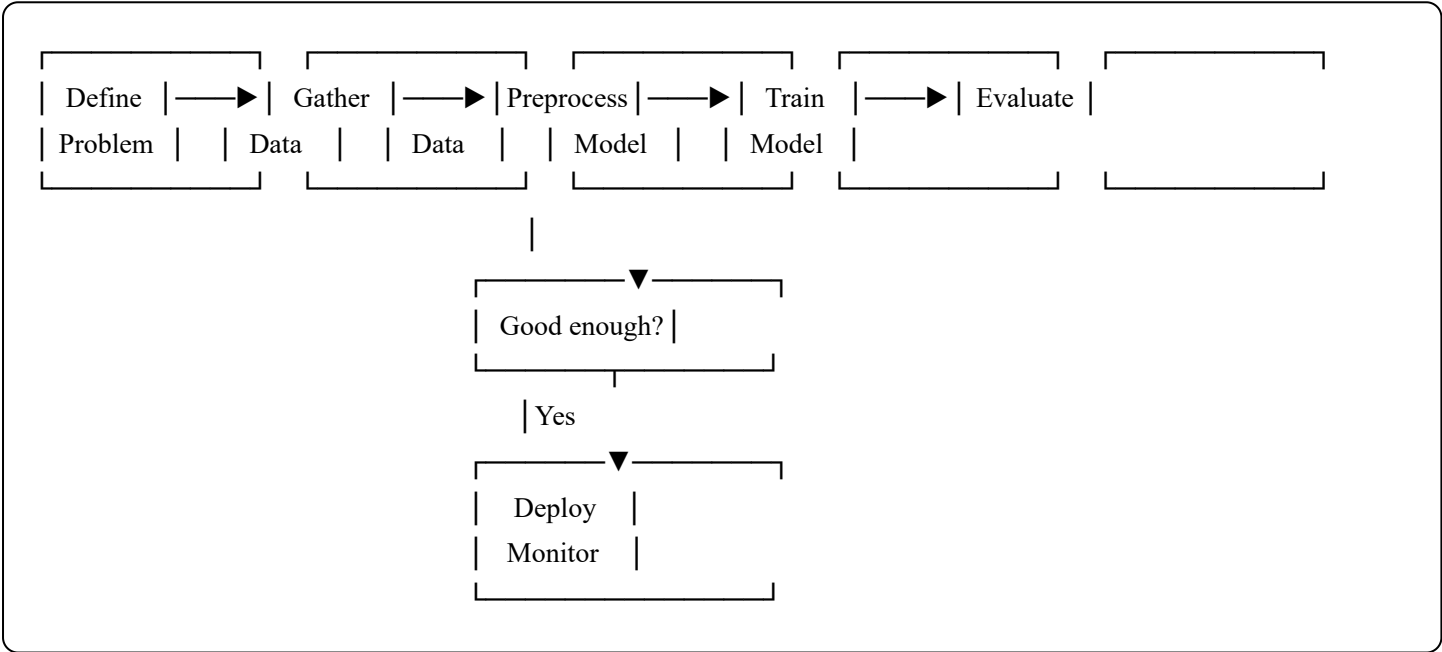


Key Terminology

Term	Meaning
Feature (X)	Input variable used for prediction
Label (y)	Output/target variable to predict
Model	Mathematical function mapping $X \rightarrow y$
Training	Process of learning parameters from data

Term	Meaning
Inference	Using a trained model to make predictions
Epoch	One full pass through the training data
Batch	Subset of training data used in one update
Parameters	Internal values learned by the model (weights, biases)
Hyperparameters	External settings you configure before training

2. The ML Workflow



Step-by-Step Breakdown

- Step 1 — Problem Definition:** Is it regression, classification, or clustering? What's the success metric?
- Step 2 — Data Collection:** Gather relevant, representative data. More high-quality data usually beats a better algorithm.
- Step 3 — Preprocessing:**
- Handle missing values (impute or drop)
 - Encode categorical variables (one-hot, label encoding)
 - Scale features (normalization, standardization)
 - Split into Train / Validation / Test sets (typically 70/15/15 or 80/20)
- Step 4 — Model Training:** Choose an algorithm, feed training data, optimize parameters.
- Step 5 — Evaluation:** Measure performance on held-out test data using appropriate metrics.

Step 6 — Deployment & Monitoring: Serve model in production; watch for data drift.

3. Key Math Foundations

Linear Algebra Essentials

Dot Product:

$$a \cdot b = a_1b_1 + a_2b_2 + \dots + a_nb_n$$

Matrix Multiplication:

If A is $(m \times k)$ and B is $(k \times n)$, then $C = A \cdot B$ is $(m \times n)$

$$C_{ij} = \sum_k A_{ik} \cdot B_{kj}$$

Transpose: $(AB)^T = B^T A^T$

Calculus: Gradient Descent

The workhorse of ML optimization. We want to **minimize a loss function $L(\theta)$** .

Gradient Descent Update Rule:

$$\theta := \theta - \alpha \cdot \nabla L(\theta)$$

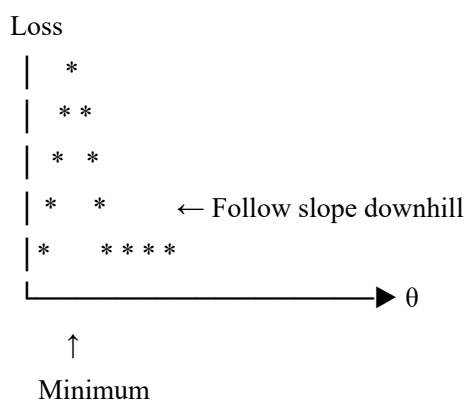
Where:

θ = model parameters

α = learning rate (step size)

$\nabla L(\theta)$ = gradient of loss w.r.t. θ

Visual intuition:



Variants:

- **Batch GD:** Use all data → stable but slow
- **Stochastic GD (SGD):** Use 1 sample → fast but noisy

- **Mini-batch GD:** Use small batch (32–256) → best of both

Probability Essentials

Bayes' Theorem:

$$P(A|B) = P(B|A) \cdot P(A) / P(B)$$

$$\text{Posterior} = \text{Likelihood} \times \text{Prior} / \text{Evidence}$$

Expected Value: $E[X] = \sum x_i \cdot P(x_i)$

Variance: $\text{Var}(X) = E[(X - \mu)^2]$

Normal Distribution:

$$f(x) = (1/\sigma\sqrt{2\pi}) \cdot \exp(-(x-\mu)^2/2\sigma^2)$$

4. Linear Regression

Theory

Linear Regression models the **relationship between a continuous output and one or more inputs** by fitting a straight line (or hyperplane in higher dimensions). It assumes a linear relationship between features and the target, with Gaussian noise.

Use when: Predicting house prices, stock values, temperature, any continuous quantity.

Assumptions:

- Linearity between X and y
- Independence of errors
- Homoscedasticity (constant variance of errors)
- Normally distributed residuals

Math

Model:

$$\begin{aligned}\hat{y} &= \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n \\ &= X\theta \text{ (in matrix form)}\end{aligned}$$

Loss Function — Mean Squared Error:

$$\text{MSE} = (1/m) \sum_i (y_i - \hat{y}_i)^2$$

Closed-Form Solution (Normal Equation):

$$\theta^* = (X^T X)^{-1} X^T y$$

Works perfectly for small datasets, but inverting $X^T X$ is $O(n^3)$ — expensive for large n .

Gradient Descent Solution:

$$\begin{aligned} \partial \text{MSE} / \partial \theta_j &= (2/m) \sum_i (\hat{y}_i - y_i) \cdot x_{ij} \\ \theta_j &:= \theta_j - \alpha \cdot (\partial \text{MSE} / \partial \theta_j) \end{aligned}$$

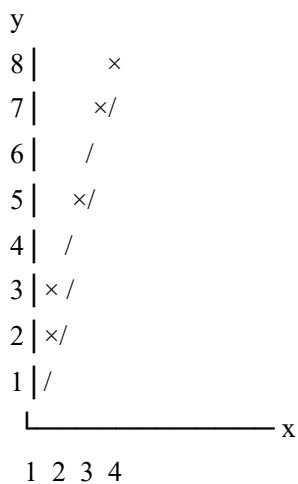
Example

Data:

x	y
1	2.1
2	3.9
3	6.2
4	7.8

Fitted: $\hat{y} = 0.2 + 1.93x$

Prediction for $x=5$: $\hat{y} = 0.2 + 9.65 = 9.85$



5. Logistic Regression

Theory

Despite the name, Logistic Regression is a **classification algorithm**. It models the probability that an input

belongs to a class. The key insight: we apply a sigmoid function to a linear combination of features to squash output to $[0, 1]$.

Use when: Binary classification — spam/not spam, disease/no disease, churn prediction.

Decision boundary is where the model is 50% confident — a hyperplane in feature space.

Math

Sigmoid Function:

$$\sigma(z) = 1 / (1 + e^{-z})$$

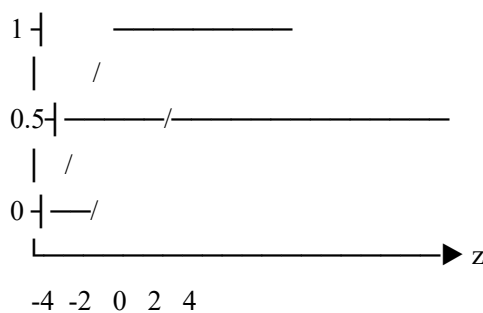
Properties:

$$\sigma(0) = 0.5$$

$$\sigma(-\infty) = 0$$

$$\sigma(+\infty) = 1$$

Sigmoid Shape:



Model:

$$P(y=1|X) = \sigma(X\theta) = 1 / (1 + \exp(-X\theta))$$

Loss — Binary Cross-Entropy:

$$L = -(1/m) \sum_i [y_i \log(\hat{y}_i) + (1-y_i) \log(1-\hat{y}_i)]$$

Why not MSE? Because cross-entropy is convex for logistic regression → guaranteed global minimum.

Prediction:

$$\hat{y} = 1 \text{ if } \sigma(X\theta) \geq 0.5$$

$$\hat{y} = 0 \text{ otherwise}$$

Multiclass Extension

Softmax extends logistic regression to K classes:

$$P(y=k|X) = \exp(X\theta_k) / \sum_j \exp(X\theta_j)$$

6. Decision Trees

Theory

Decision Trees make predictions by **learning a hierarchy of if-then-else rules** from data. Each internal node tests a feature, each branch represents an outcome, and each leaf gives a prediction. Extremely interpretable — you can print the rules.

Strength: No feature scaling needed, handles mixed types, interpretable.

Weakness: Prone to overfitting, unstable (high variance).

How Splits Are Chosen

The algorithm greedily picks the feature and threshold that maximally **reduces impurity** at each node.

Gini Impurity (Classification):

$$\text{Gini} = 1 - \sum_k p_k^2$$

Where p_k = fraction of samples belonging to class k

Gini = 0 → perfectly pure node

Gini = 0.5 → maximally impure (50/50 binary split)

Information Gain (Entropy-based):

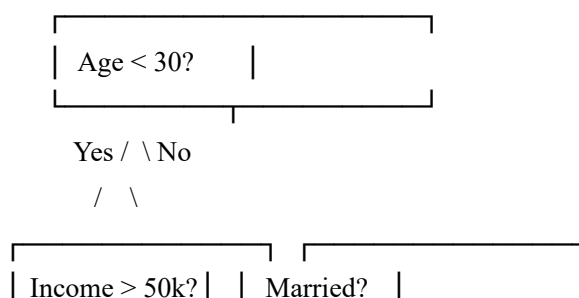
$$\text{Entropy } H(S) = -\sum_k p_k \log_2(p_k)$$

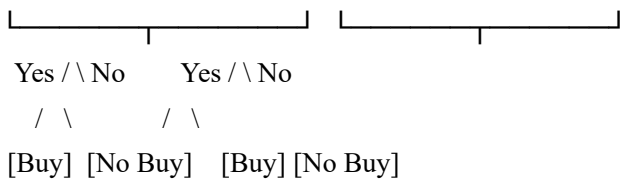
$$\text{Information Gain} = H(\text{parent}) - \sum_j (|S_j|/|S|) \cdot H(S_j)$$

For Regression — Variance Reduction:

$$\text{Score} = \text{Var}(\text{parent}) - \sum_j (|S_j|/|S|) \cdot \text{Var}(S_j)$$

Tree Diagram Example





Key Hyperparameters

Parameter	Effect
max_depth	Limits tree depth; prevents overfitting
min_samples_split	Min samples needed to split a node
min_samples_leaf	Min samples in a leaf node
max_features	Features considered per split

7. Ensemble Methods

Theory

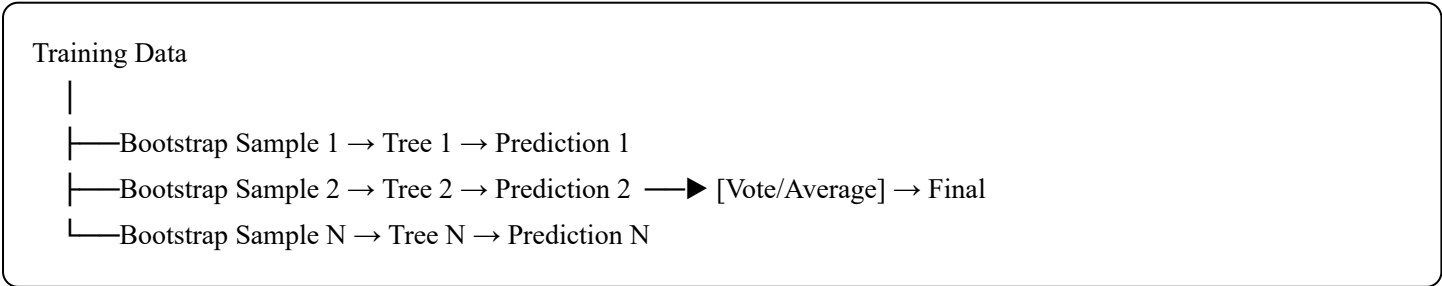
Ensemble methods **combine multiple weaker models** to produce a stronger one. The intuition: if each model makes different errors, averaging them out reduces overall error. The diversity of base models is key.

Three main strategies:

- 1. **Bagging** — train models in parallel on bootstrapped data subsets
- 2. **Boosting** — train models sequentially, each correcting the previous
- 3. **Stacking** — use predictions of models as features for a meta-model

7.1 Random Forest (Bagging)

Trains many decision trees on random subsets of data and features, then **aggregates predictions by majority vote or averaging**.



Why better than single tree? Variance is reduced by averaging. Bias stays similar. Each tree also considers a random subset of features at each split, decorrelating trees.

Variance of average of m trees:

If trees are uncorrelated: $\text{Var}(\text{avg}) = \sigma^2/m$

If trees have correlation ρ : $\text{Var}(\text{avg}) = \rho\sigma^2 + (1-\rho)\sigma^2/m$

7.2 Gradient Boosting (Boosting)

Builds trees **sequentially**, each new tree fitting the **residuals** (errors) of the previous ensemble.

Step 1: Start with a constant prediction (e.g., mean of y)

Step 2: Compute residuals $r_i = y_i - F(x_i)$

Step 3: Fit a new tree to residuals

Step 4: Update: $F(x) := F(x) + \alpha \cdot \text{tree}(x)$

Step 5: Repeat steps 2–4 for T iterations

Update Rule:

$$F_t(x) = F_{t-1}(x) + \alpha \cdot h_t(x)$$

Where:

α = learning rate (shrinkage)

$h_t(x)$ = weak learner fitted to pseudo-residuals

Popular implementations: XGBoost, LightGBM, CatBoost

XGBoost adds:

- Regularization on tree weights
- Second-order Taylor expansion for optimization
- Column subsampling

7.3 AdaBoost

Assigns **higher weights to misclassified samples** so subsequent trees focus on hard examples.

$$\alpha_t = 0.5 \cdot \ln((1 - \epsilon_t) / \epsilon_t)$$

$$w_i \leftarrow w_i \cdot \exp(-\alpha_t y_i h_t(x_i))$$

Where ϵ_t = weighted error rate of tree t

8. Support Vector Machines

Theory

SVMs find the **maximum-margin hyperplane** that separates two classes. The intuition: the further data points are from the decision boundary, the more confident the classification, and the better the generalization.

Points closest to the decision boundary are called **support vectors** — they alone define the hyperplane. This makes SVMs robust to outliers far from the boundary.

Key idea — Kernel Trick: Map data to higher dimensions where it becomes linearly separable, without explicitly computing the transformation.

Math

Decision boundary (hyperplane):

$$w \cdot x + b = 0$$

$$\text{Class +1: } w \cdot x + b \geq +1$$

$$\text{Class -1: } w \cdot x + b \leq -1$$

Margin:

$$\text{Margin} = 2 / \|w\|$$

$$\text{Maximize margin} \leftrightarrow \text{Minimize } \|w\|^2$$

Optimization (Hard Margin):

$$\text{Minimize: } (1/2)\|w\|^2$$

$$\text{Subject to: } y_i(w \cdot x_i + b) \geq 1 \text{ for all } i$$

Soft Margin (C-SVM):

$$\text{Minimize: } (1/2)\|w\|^2 + C \sum_i \xi_i$$

$$\text{Subject to: } y_i(w \cdot x_i + b) \geq 1 - \xi_i, \xi_i \geq 0$$

C = tradeoff: large C → small margin, fewer violations

small C → large margin, more violations allowed

Common Kernels:

Linear: $K(x,z) = x \cdot z$

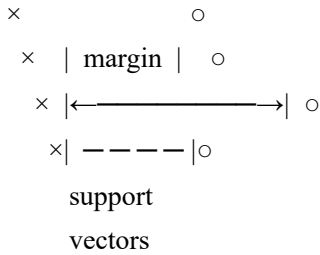
Polynomial: $K(x,z) = (x \cdot z + c)^d$

RBF/Gaussian: $K(x,z) = \exp(-\gamma \|x-z\|^2)$

Sigmoid: $K(x,z) = \tanh(\alpha x \cdot z + c)$

Visual — Margin

Class A (×) Class B (○)



9. K-Nearest Neighbors

Theory

KNN is a **non-parametric, instance-based** algorithm. It memorizes the entire training set. To predict, it finds the K closest training points and takes their majority vote (classification) or average (regression).

No training phase — computation happens at prediction time. This makes it slow for large datasets but simple and effective for small ones.

K selection:

- Small K → Complex boundary, overfitting
- Large K → Smooth boundary, underfitting
- Use cross-validation to find optimal K

Math

Distance Metrics:

Euclidean: $d(x,z) = \sqrt{\sum_i (x_i - z_i)^2}$

Manhattan: $d(x,z) = \sum_i |x_i - z_i|$

Minkowski: $d(x,z) = (\sum_i |x_i - z_i|^p)^{1/p}$

Cosine: $d(x,z) = 1 - (x \cdot z) / (\|x\| \cdot \|z\|)$

Prediction (classification):

$$\hat{y} = \text{mode}(\{y(x_i) : x_i \in N_k(x)\})$$

Where $N_k(x)$ = K nearest neighbors of x

Visual

New point: ★

K=3 neighbors: ×, ×, ○ → Predict ×

K=5 neighbors: ×, ×, ○, ×, ○ → Predict ×

```

×  ×
  ★
×  ○
×  ○
  ○

```

Important: Always normalize/scale features before using KNN, because distance is sensitive to feature magnitude.

10. Naive Bayes

Theory

Naive Bayes is a **probabilistic classifier** based on Bayes' theorem. The "naive" assumption is that **features are conditionally independent** given the class label. While this is rarely true in practice, it often works surprisingly well, especially for text classification.

Why it works: Even if the probability estimates are off, the **ranking of classes** is often correct for classification purposes.

Variants: Gaussian (continuous features), Multinomial (word counts), Bernoulli (binary features).

Math

Bayes' Theorem Applied:

$$P(y|X) = P(X|y) \cdot P(y) / P(X)$$

With naive independence assumption:

$$P(X|y) = P(x_1|y) \cdot P(x_2|y) \cdot \dots \cdot P(x_n|y)$$

Final classifier:

$$\hat{y} = \text{argmax}_y P(y) \cdot \prod_i P(x_i|y)$$

In log space (numerically stable):

$$\hat{y} = \operatorname{argmax}_y [\log P(y) + \sum_i \log P(x_i|y)]$$

Gaussian Naive Bayes:

$$P(x_i|y) = (1/\sigma_{y_i}\sqrt{2\pi}) \cdot \exp(-(x_i - \mu_{y_i})^2 / 2\sigma_{y_i}^2)$$

Laplace Smoothing (prevents zero probability):

$$P(x_i=v|y) = (\text{count}(x_i=v, y) + \alpha) / (\text{count}(y) + \alpha \cdot |V|)$$

Where $\alpha = 1$ (Laplace), $|V|$ = vocabulary size

11. Clustering (Unsupervised)

11.1 K-Means

Theory: K-Means partitions data into K groups by iteratively assigning points to the nearest centroid and updating centroids. It minimizes **within-cluster sum of squares (WCSS)**.

Algorithm:

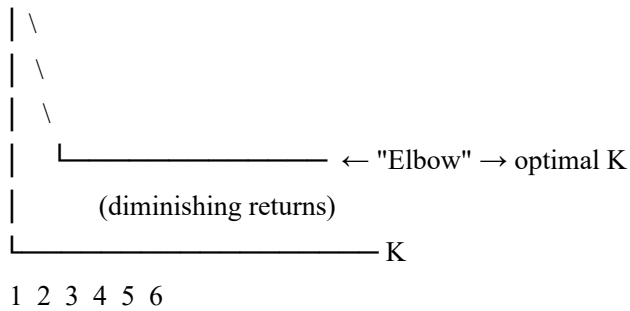
1. Initialize K centroids (randomly or K-Means++)
2. Assign each point to nearest centroid:
$$c_i = \operatorname{argmin}_k \|x_i - \mu_k\|^2$$
3. Update centroids:
$$\mu_k = (1/|C_k|) \sum_{i \in C_k} x_i$$
4. Repeat steps 2–3 until convergence

Objective (WCSS):

$$J = \sum_k \sum_{i \in C_k} \|x_i - \mu_k\|^2$$

Choosing K — Elbow Method:

WCSS



11.2 DBSCAN

Theory: Density-Based Spatial Clustering. Groups points that are closely packed together and marks outliers as noise. Unlike K-Means, it finds **arbitrary-shaped clusters** and automatically detects the number of clusters.

Key concepts:

- **Core point:** Has $\geq \text{minPts}$ neighbors within radius ϵ
- **Border point:** Within ϵ of a core point but not core itself
- **Noise:** Neither core nor border

Parameters: ϵ (radius), minPts (minimum neighbors)

ϵ

• = Core point

○ = Border point

× = Noise

×

×

•

•

11.3 Hierarchical Clustering

Builds a **tree of clusters (dendrogram)** by successively merging (agglomerative) or splitting (divisive) clusters.

Linkage methods:

- Single: min distance between clusters
- Complete: max distance between clusters
- Average: mean distance
- Ward: minimizes within-cluster variance

12. Dimensionality Reduction

12.1 Principal Component Analysis (PCA)

Theory: PCA finds the directions of **maximum variance** in data and projects onto a lower-dimensional subspace. The new axes (principal components) are orthogonal linear combinations of original features.

Why reduce dimensions?

- Remove noise and redundancy
- Visualization (project to 2D/3D)
- Speed up downstream models
- Combat the curse of dimensionality

Algorithm:

1. Standardize data (zero mean, unit variance)
2. Compute covariance matrix: $\Sigma = (1/m) X^T X$
3. Compute eigenvectors and eigenvalues of Σ
4. Sort eigenvectors by eigenvalue (descending)
5. Project data: $Z = X \cdot W$ (W = top-k eigenvectors)

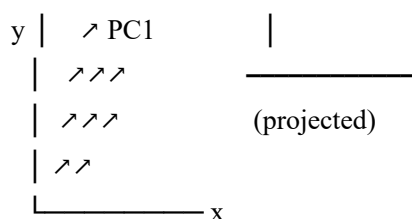
Explained Variance:

Explained ratio for PC_k = $\lambda_k / \sum_i \lambda_i$

Choose k such that cumulative explained variance $\geq 95\%$

Visual:

Original 2D data: After PCA (1D):



12.2 t-SNE

Theory: t-Distributed Stochastic Neighbor Embedding is a **non-linear** dimensionality reduction technique primarily used for **visualization** of high-dimensional data. It preserves local structure — nearby points in high-D stay nearby in low-D.

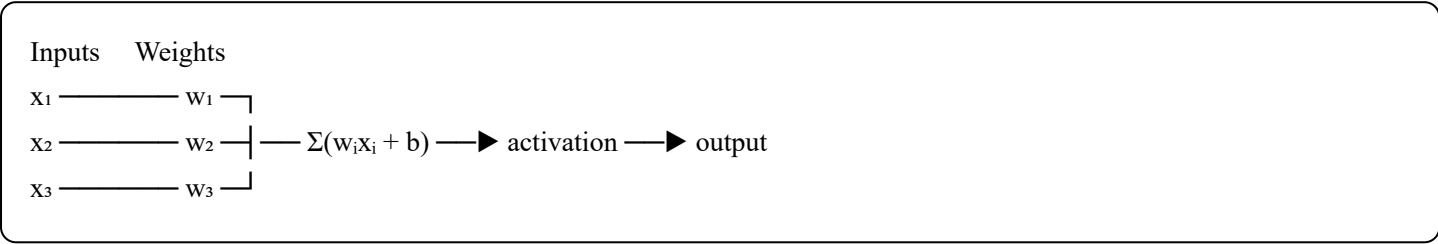
Key idea: Model similarity as probability distributions, then minimize KL divergence between high-D and low-D distributions.

⚠️ t-SNE is for visualization only — distances between clusters are not meaningful, and you cannot project new points without rerunning.

13. Neural Networks & Deep Learning

13.1 The Neuron

A single artificial neuron computes a weighted sum of inputs, then applies an activation function:

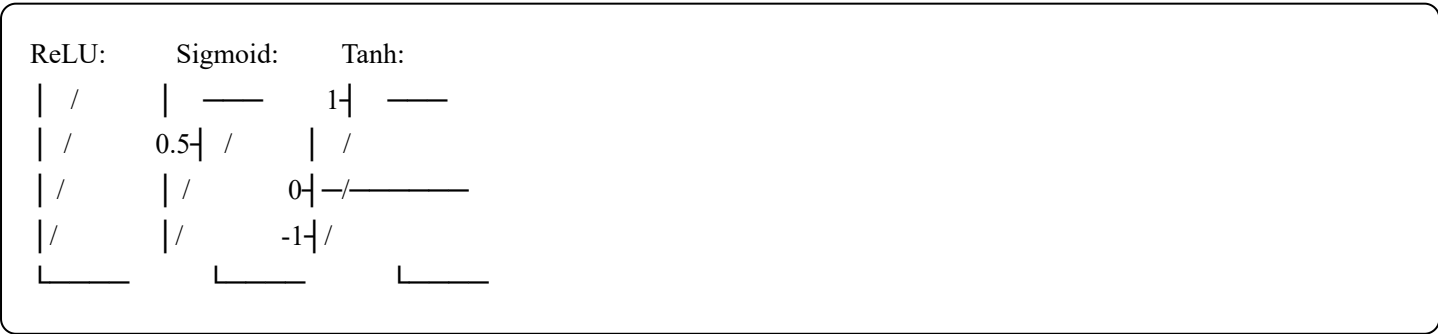


Math:

$$z = w \cdot x + b = \sum_i w_i x_i + b$$
$$a = \sigma(z) \text{ (apply activation function)}$$

13.2 Activation Functions

Function	Formula	Use Case
Sigmoid	$1/(1+e^{-z})$	Binary output, old RNNs
Tanh	$(e^z - e^{-z}) / (e^z + e^{-z})$	Hidden layers (zero-centered)
ReLU	$\max(0, z)$	Hidden layers (most common)
Leaky ReLU	$\max(0.01z, z)$	Fixes dying ReLU
Softmax	$\exp(z_k) / \Sigma \exp(z_j)$	Multiclass output



13.3 Neural Network Architecture



Pooling: Downsamples feature maps. Max pooling takes the max in each window.

Architecture:

Input → Conv → ReLU → Pool → Conv → ReLU → Pool → Flatten → Dense → Output

13.7 LSTM (Long Short-Term Memory)

Solves the **vanishing gradient problem** in vanilla RNNs via gating mechanisms.

Three gates control information flow:

Forget gate: $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$ ← What to forget

Input gate: $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$ ← What to store

Output gate: $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$ ← What to output

Cell state: $C_t = f_t \odot C_{t-1} + i_t \odot \tanh(WC[h_{t-1}, x_t] + bC)$

Hidden state: $h_t = o_t \odot \tanh(C_t)$

14. Model Evaluation & Metrics

Classification Metrics

Confusion Matrix:

	Predicted	
	Positive	Negative
Actual Pos	TP	FN
Actual Neg	FP	TN

Core Metrics:

Accuracy = $(TP + TN) / (TP + TN + FP + FN)$

Precision = $TP / (TP + FP)$ ← Of predicted positives, how many correct?

Recall = $TP / (TP + FN)$ ← Of actual positives, how many found?

F1 Score = $2 \cdot (P \cdot R) / (P + R)$ ← Harmonic mean of precision and recall

When to use which:

- **Accuracy:** Balanced classes
- **Precision:** False positives are costly (spam detection)
- **Recall:** False negatives are costly (cancer detection)
- **F1:** Imbalanced classes, want balance of P and R

ROC-AUC:

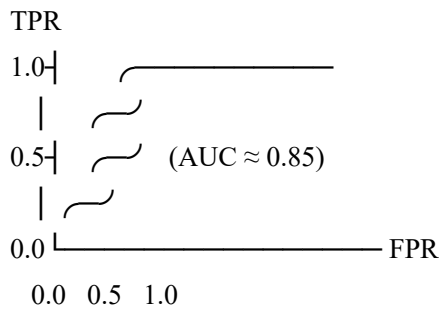
ROC curve: TPR vs FPR at different thresholds

$$\text{TPR} = \text{Recall} = \text{TP}/(\text{TP}+\text{FN})$$

$$\text{FPR} = \text{FP}/(\text{FP}+\text{TN})$$

AUC = 0.5 → random classifier

AUC = 1.0 → perfect classifier



Regression Metrics

$$\text{MAE} = (1/m) \sum |y_i - \hat{y}_i| \quad \leftarrow \text{Mean Absolute Error}$$

$$\text{MSE} = (1/m) \sum (y_i - \hat{y}_i)^2 \quad \leftarrow \text{Mean Squared Error}$$

$$\text{RMSE} = \sqrt{\text{MSE}} \quad \leftarrow \text{Same units as } y$$

$$\begin{aligned} R^2 &= 1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}} \quad \leftarrow 1 = \text{perfect}, 0 = \text{mean baseline} \\ &= 1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2 \end{aligned}$$

Cross-Validation

K-Fold CV: Split data into K folds; train on K-1, test on 1, rotate K times.

K=5:

Fold 1: [Test] [Train] [Train] [Train] [Train]

Fold 2: [Train] [Test] [Train] [Train] [Train]

Fold 3: [Train] [Train] [Test] [Train] [Train]

...

Final score = mean of 5 test scores

Stratified K-Fold: Ensures class distribution is preserved in each fold.

LOOCV: K=n (each sample is a test set once); expensive but useful for small data.

15. Regularization

Theory

Regularization adds a **penalty term to the loss function** to discourage model complexity, thereby reducing overfitting. It constrains the model's parameters from growing too large.

Core idea: Add complexity cost to training so the model generalizes better.

L1 (Lasso) Regularization

$$\text{Loss} = \text{MSE} + \lambda \sum_i |\theta_i|$$

Effect: Drives many weights to exactly zero $\rightarrow$ sparse model $\rightarrow$ feature selection

Use when: You suspect few features actually matter

L2 (Ridge) Regularization

$$\text{Loss} = \text{MSE} + \lambda \sum_i \theta_i^2$$

Effect: Shrinks all weights toward zero but rarely to zero exactly

Use when: All features may be relevant; want to prevent large weights

Elastic Net

$$\text{Loss} = \text{MSE} + \lambda_1 \sum |\theta_i| + \lambda_2 \sum \theta_i^2$$

Combines L1 and L2; balances sparsity and weight shrinkage

Dropout (Neural Networks)

Randomly **zeros out neurons** during training with probability p . Forces the network to learn redundant representations.

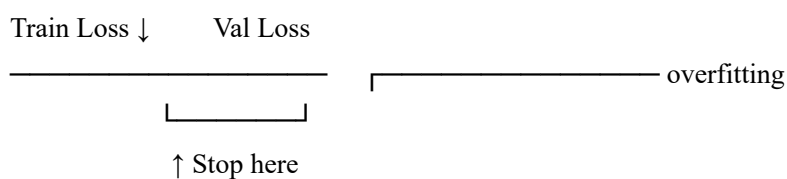
Training: Each neuron active with prob $(1-p)$

Inference: All neurons active, weights scaled by $(1-p)$

Common: $p=0.5$ for hidden layers, $p=0.2$ for input layer

Early Stopping

Monitor validation loss during training; stop when it starts increasing.



16. Feature Engineering

Theory

Feature engineering is the process of **transforming raw data into informative features** that better represent the underlying problem to the ML model. Often more impactful than algorithm choice.

Feature Scaling

Standardization (Z-score):

$$x' = (x - \mu) / \sigma$$

→ Output: mean=0, std=1

→ Use when: Normal distribution, SVM, PCA, neural networks

Min-Max Normalization:

$$x' = (x - x_{\min}) / (x_{\max} - x_{\min})$$

→ Output: [0, 1]

→ Use when: Neural networks, KNN (bounded range known)

Robust Scaling:

$$x' = (x - \text{median}) / \text{IQR}$$

→ Robust to outliers

Encoding Categorical Variables

Method	When	Example
Label Encoding	Ordinal categories	Low=0, Med=1, High=2
One-Hot Encoding	Nominal, low cardinality	Color → [R, G, B] flags
Target Encoding	High cardinality	Replace category with mean(y)
Embedding	Neural networks, NLP	Map to dense vector

Handling Missing Values

- **Drop:** Remove rows/columns with > 40–50% missing
- **Mean/Median Imputation:** Fill with column statistic
- **Mode Imputation:** For categorical
- **KNN Imputation:** Fill using similar rows
- **Model-based:** Predict missing values with another model

Feature Creation

- **Polynomial features:** $x_1, x_2 \rightarrow x_1^2, x_2^2, x_1x_2$
 - **Log transform:** Reduces skew for right-skewed features
 - **Interaction terms:** Combine two features
 - **Date features:** Extract day, month, year, day-of-week, is_weekend
 - **Aggregations:** Mean, std, min, max per group (for tabular data)
-

17. Bias-Variance Tradeoff

Theory

The **fundamental tension in machine learning**: you want a model complex enough to capture true patterns, but simple enough not to memorize noise.

Total Error = Bias² + Variance + Irreducible Noise

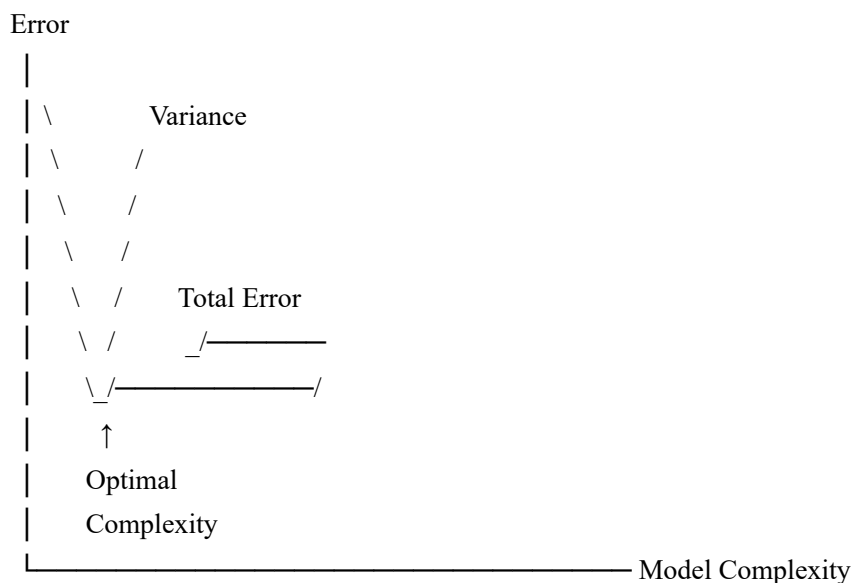
Bias: Error from wrong assumptions (underfitting)

→ Model too simple, misses patterns

Variance: Error from sensitivity to data fluctuations (overfitting)

→ Model too complex, fits noise

Visual Representation



Diagnosis:

High Bias (underfitting):

Train Error: HIGH

Val Error: HIGH

High Variance (overfitting):

Train Error: LOW

Val Error: HIGH (large gap)

Good Fit:

Train Error: LOW-MEDIUM

Val Error: LOW-MEDIUM (small gap)

Fixes:

Problem	Fix
High Bias	More complex model, add features, reduce regularization
High Variance	More data, simpler model, regularization, dropout

18. Hyperparameter Tuning

Theory

Hyperparameters are settings you configure **before training**. Finding the best combination dramatically improves model performance.

Methods

Grid Search: Try all combinations in a defined grid.

```
python

params = {
    'learning_rate': [0.001, 0.01, 0.1],
    'max_depth': [3, 5, 7],
    'n_estimators': [100, 200, 500]
}

# Tries  $3 \times 3 \times 3 = 27$  combinations
```

→ Exhaustive but expensive for large grids.

Random Search: Sample random combinations. → Often finds good results faster than grid search.

Bayesian Optimization: Use a probabilistic model to select next hyperparameter combination intelligently based on past results. → Most sample-efficient; used in Optuna, HyperOpt.

Cross-Validation Rule: Always evaluate hyperparameters on validation set, never on test set.

Important Hyperparameters by Algorithm

- Linear/Logistic Regression: C (regularization strength)
- Decision Tree: max_depth, min_samples_split
- Random Forest: n_estimators, max_features, max_depth
- Gradient Boosting: learning_rate, n_estimators, max_depth, subsample
- SVM: C, kernel, gamma
- KNN: K (n_neighbors), distance metric
- Neural Network: learning_rate, batch_size, layers, neurons, dropout, epochs

19. Algorithm Comparison Table

Algorithm	Type	Interpretable	Scales	Handles NaN	Feature Scaling	Typical Use
Linear Reg	Supervised	✓	✓	✗	Needed	Continuous prediction
Logistic Reg	Supervised	✓	✓	✗	Needed	Binary classification
Decision Tree	Supervised	✓	✗	✗	✗	Interpretable rules
Random Forest	Supervised	⚠	⚠	✗	✗	High accuracy tabular
Gradient Boost	Supervised	✗	⚠	✓	✗	Competition winner
SVM	Supervised	✗	✗	✗	Needed	High-dim, small data
KNN	Supervised	⚠	✗	✗	Needed	Simple baseline
Naive Bayes	Supervised	✓	✓	✗	✗	Text classification
Neural Net	Supervised	✗	✓	✗	Needed	Images, NLP, complex
K-Means	Unsupervised	⚠	✓	✗	Needed	Customer segmentation
DBSCAN	Unsupervised	✗	⚠	✗	Needed	Anomaly detection
PCA	Reduction	⚠	✓	✗	Needed	Visualization, noise

20. Quick Reference Formulas

Loss Functions

REGRESSION	
MSE = $(1/m)\Sigma(y - \hat{y})^2$	
MAE = $(1/m)\Sigma y - \hat{y} $	
Huber = MSE if $ y - \hat{y} \leq \delta$, else $\delta y - \hat{y} - \delta^2/2$	
CLASSIFICATION	
Binary CE = $-[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$	
Categorical = $-\Sigma_k y_k \cdot \log(\hat{y}_k)$	
Hinge = $\max(0, 1 - y \cdot f(x))$	

Optimization

SGD: $\theta := \theta - \alpha \nabla L$
Momentum: $v := \beta v - \alpha \nabla L; \theta := \theta + v$
AdaGrad: $G := G + (\nabla L)^2; \theta := \theta - (\alpha / \sqrt{G}) \cdot \nabla L$
Adam: $m := \beta_1 m + (1 - \beta_1) \nabla L$
$v := \beta_2 v + (1 - \beta_2) (\nabla L)^2$
$\theta := \theta - \alpha \cdot \hat{m} / \sqrt{\hat{v}}$ (bias-corrected)

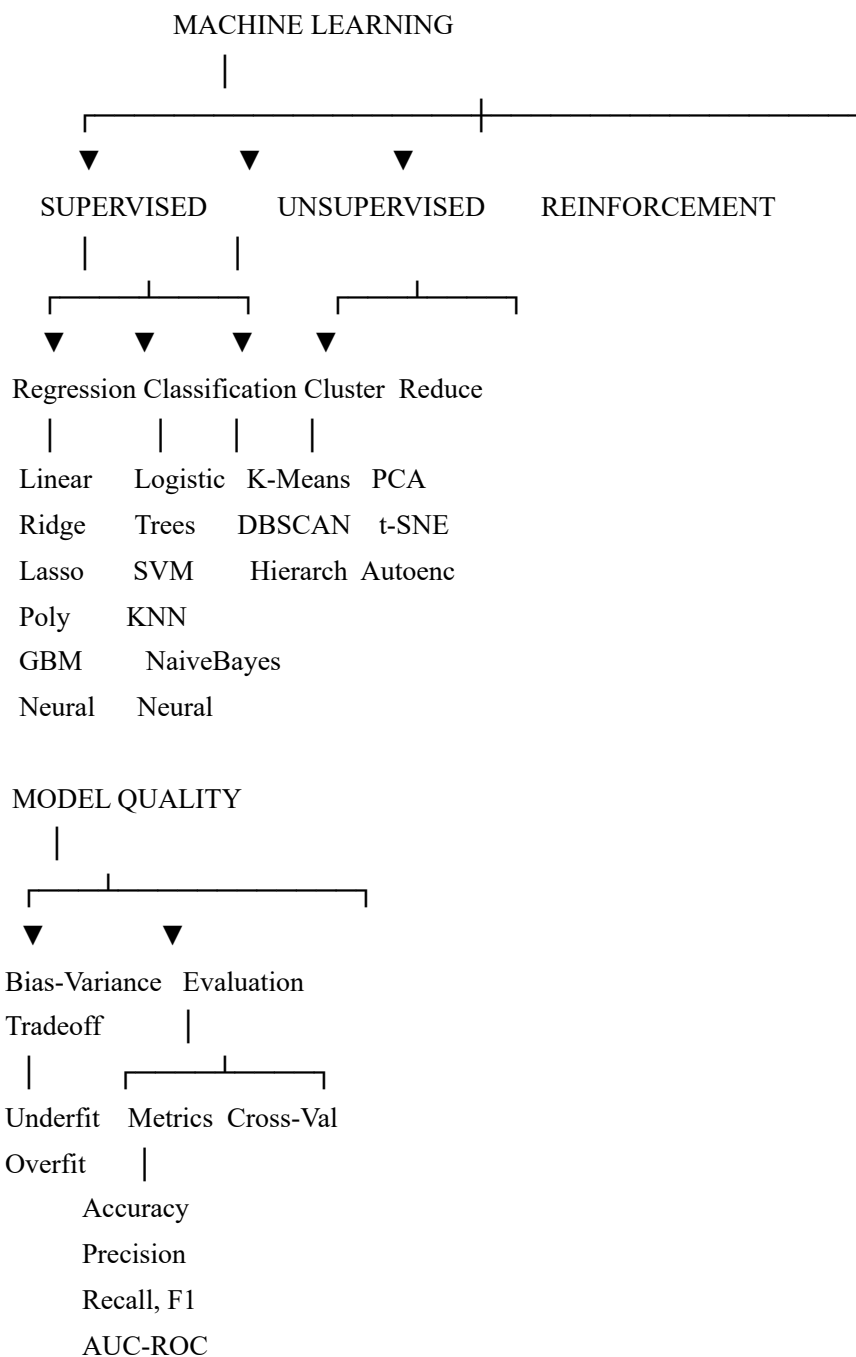
Key Decision Guide

Problem Type → Algorithm Choice:
Is your output continuous?
Yes → REGRESSION
Few features, interpretable → Linear Regression
Complex patterns → Gradient Boosting, Neural Net
Is your output categorical?
Yes → CLASSIFICATION
Linearly separable → Logistic Regression, SVM
Non-linear → Random Forest, Gradient Boosting
Text → Naive Bayes, Transformer
Images → CNN
No labels?
Yes → UNSUPERVISED
Find groups → K-Means, DBSCAN

Reduce dims → PCA, t-SNE

Detect anomalies → Isolation Forest, Autoencoder

📖 ML Concept Mind Map



📌 Golden Rules of ML

1. **Start simple.** A logistic regression or linear model is your baseline — always.
2. **Data > Algorithm.** More quality data beats a fancy model on bad data.
3. **Validate properly.** Never evaluate on training data; use held-out test set.

4. **Scale your features.** Most algorithms except trees need scaled inputs.
 5. **Understand your metric.** Accuracy is misleading on imbalanced data.
 6. **Regularize.** When in doubt, add L2 regularization.
 7. **Ensembles win.** Combining models almost always beats a single model.
 8. **Feature engineering matters.** Transforming your features intelligently often beats hyperparameter tuning.
 9. **Monitor in production.** Models degrade as data distribution shifts.
 10. **Iterate.** Build fast, evaluate honestly, improve incrementally.
-

21. Transformers & Attention Mechanism

Theory

Transformers, introduced in the landmark 2017 paper "*Attention Is All You Need*", replaced recurrent architectures for sequence tasks. The key insight: instead of processing tokens one-by-one (like RNNs), transformers **process all tokens simultaneously** and learn which tokens should "attend" to which others — regardless of distance.

This parallelism makes transformers vastly faster to train and enables them to capture long-range dependencies that RNNs struggle with. Today, transformers underpin virtually all state-of-the-art NLP (BERT, GPT, T5) and increasingly vision (ViT) and multimodal systems.

Self-Attention: The Core Idea

Each token creates three vectors from its embedding — **Query (Q)**, **Key (K)**, and **Value (V)**. Attention weights are computed as how much each token's query matches every other token's key.

For each token i :

$Q_i = W_Q \cdot x_i$ (what am I looking for?)

$K_i = W_K \cdot x_i$ (what do I offer?)

$V_i = W_V \cdot x_i$ (what do I actually contain?)

Scaled Dot-Product Attention:

$\text{Attention}(Q, K, V) = \text{softmax}(QK^T / \sqrt{d_k}) \cdot V$

Where:

QK^T = raw attention scores (how much each token attends to each other)

$\sqrt{d_k}$ = scaling factor (prevents vanishing gradients for large d_k)

softmax = converts scores to probabilities summing to 1

$\cdot V$ = weighted sum of values

Attention Matrix (example, 4 tokens):

"cat" "sat" "on" "mat"

"cat" [[0.6, 0.2, 0.1, 0.1],

"sat" [0.1, 0.5, 0.3, 0.1],

"on" [0.1, 0.2, 0.4, 0.3],

"mat" [0.2, 0.1, 0.2, 0.5]]

Diagonal dominance = token attends to itself

Off-diagonal = cross-token relationships

Multi-Head Attention

Run h attention functions in parallel, then concatenate:

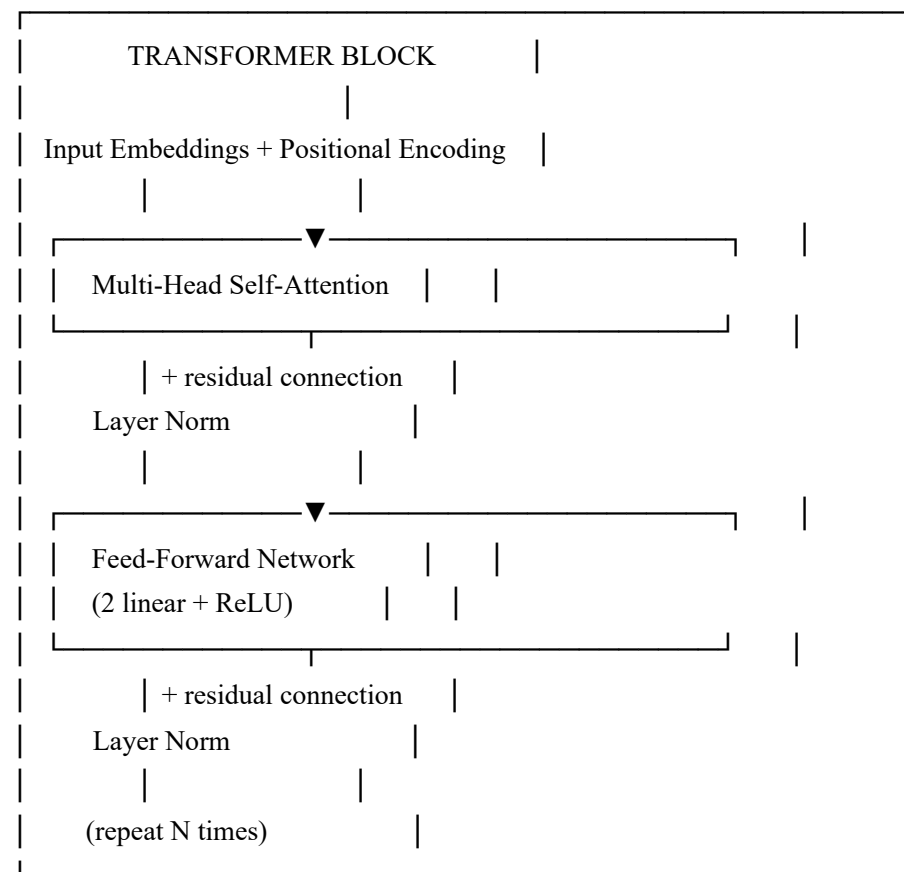
$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W_O$

Each $\text{head}_i = \text{Attention}(QW_iQ, KW_iK, VW_iV)$

Why? Different heads learn different relationship types

(syntax, coreference, positional, semantic...)

Transformer Architecture



Positional Encoding

Transformers have no inherent sense of order. Positional encodings are added to embeddings to inject sequence position information:

$$PE(pos, 2i) = \sin(pos / 10000^{(2i/d_model)})$$
$$PE(pos, 2i+1) = \cos(pos / 10000^{(2i/d_model)})$$

BERT vs GPT Architecture Variants

	BERT	GPT	
Type	Encoder-only	Decoder-only	
Attention	Bidirectional	Causal (left-to-right)	
Pre-training	Masked LM	Next token predict	
Best for	Classification	Generation	
	Q&A, NER	Chat, completion	

22. Reinforcement Learning

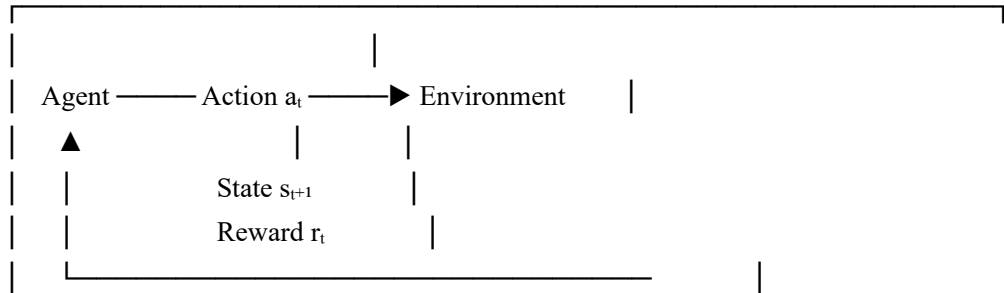
Theory

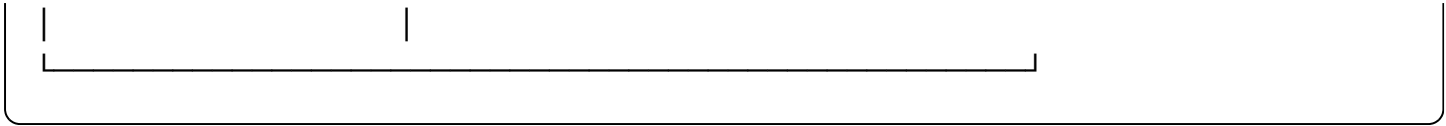
Reinforcement Learning (RL) is a learning paradigm where an **Agent** interacts with an **Environment**, taking **Actions** to maximize cumulative **Reward** over time. Unlike supervised learning, there are no labeled examples — the agent must discover good strategies through trial and error.

RL is inspired by behavioral psychology: actions that lead to rewards are reinforced, while those that don't are discouraged.

Applications: Game playing (AlphaGo, Atari), robotics, autonomous driving, recommendation systems, resource management, RLHF for LLMs.

Core Components





Term	Meaning
State (s)	Current situation of the agent
Action (a)	Choice the agent makes
Reward (r)	Feedback signal from environment
Policy (π)	Agent's strategy: maps states $\rightarrow$ actions
Value function V(s)	Expected cumulative reward from state s
Q-function Q(s,a)	Expected cumulative reward from (s,a)

Key Equations

Return (discounted cumulative reward):

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_k \gamma^k r_{t+k}$$

γ = discount factor ($0 < \gamma \leq 1$)

γ near 0 $\rightarrow$ myopic (values immediate reward)

γ near 1 $\rightarrow$ far-sighted (values future reward)

Bellman Equation (V-function):

$$V(s) = E[r + \gamma V(s') \mid s]$$
$$= \sum_a \pi(a|s) \sum_{s'} P(s'|s,a) [r(s,a,s') + \gamma V(s')]$$

Bellman Equation (Q-function):

$$Q(s,a) = E[r + \gamma \max_{a'} Q(s',a')]$$

Q-Learning (Tabular)

Update rule:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Where:

α = learning rate

$$r + \gamma \max Q(s', a') = \text{TD target}$$

$$r + \gamma \max Q - Q(s, a) = \text{TD error } (\delta)$$

Deep Q-Network (DQN)

Replace Q-table with a neural network $Q(s, a; \theta)$. Key innovations:

- **Experience replay:** Store (s, a, r, s') in buffer, sample mini-batches
- **Target network:** Separate frozen network for TD targets (updated periodically)

$$\text{Loss} = E[(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta))^2]$$

TD target (frozen)

Policy Gradient (REINFORCE)

Directly optimize the policy $\pi(a|s; \theta)$ by gradient ascent on expected return:

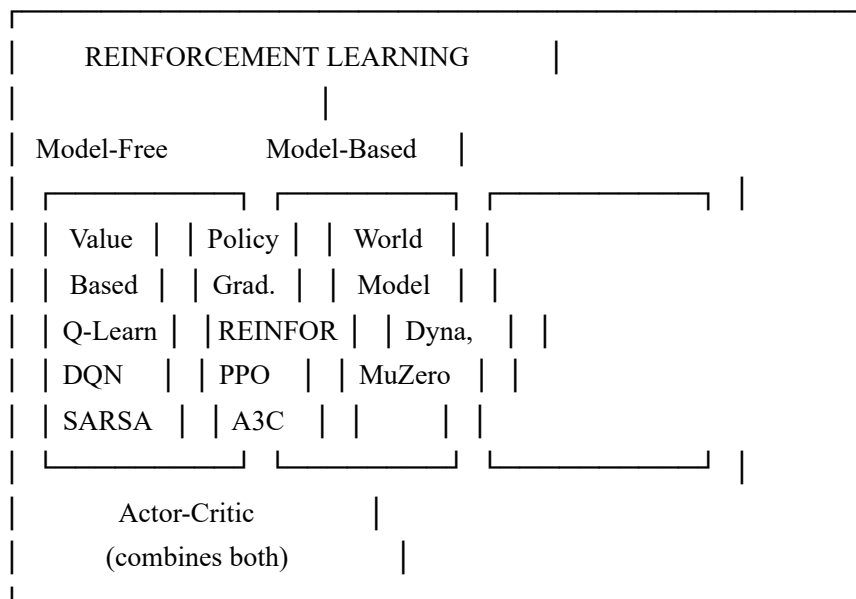
$$\nabla_{\theta} J(\theta) = E[G_t \cdot \nabla_{\theta} \log \pi(a_t | s_t; \theta)]$$

Intuition:

If $G_t > 0 \rightarrow$ increase probability of action a_t

If $G_t < 0 \rightarrow$ decrease probability of action a_t

RL Algorithm Landscape



23. Anomaly Detection

Theory

Anomaly detection identifies **rare observations that deviate significantly** from the majority of data. Also called outlier detection. The core challenge: you usually have very few (or no) labeled anomalies to train on, so most methods are unsupervised.

Types of anomalies:

- **Point anomaly:** Single data point that is unusual (e.g., a single fraudulent transaction)
- **Contextual anomaly:** Normal value in wrong context (e.g., 30°C in winter)
- **Collective anomaly:** A sequence that is unusual together (e.g., unusual sequence of actions)

Applications: Fraud detection, intrusion detection, manufacturing defect detection, medical diagnosis, predictive maintenance.

Methods

Statistical Methods:

Z-Score:

$$z = (x - \mu) / \sigma$$

$|z| > 3 \rightarrow$ anomaly (3-sigma rule)

Assumes Gaussian distribution

IQR Method:

$$\text{IQR} = Q3 - Q1$$

Outlier if $x < Q1 - 1.5 \cdot \text{IQR}$ or $x > Q3 + 1.5 \cdot \text{IQR}$

Isolation Forest:

Builds random trees by randomly selecting a feature and a split value. Anomalies are isolated in fewer splits (shorter path length in tree).

$$\text{Score} = 2^{(-E[h(x)] / c(n))}$$

Where:

$E[h(x)]$ = average path length across trees

$c(n)$ = average path length of unsuccessful BST search

Score $\rightarrow 1$ means anomaly; $\rightarrow 0.5$ means normal

Intuition:

Normal points: require many splits to isolate (deep in tree)

Anomalies: isolated quickly (shallow in tree)

```

    Root
  /   \
Split1  Anomaly ← isolated early!
 /     \
Split2  Split3
 /\     /\
...     ...
(normal points buried deep)

```

Local Outlier Factor (LOF):

Compares local density of a point to its neighbors. Points in low-density regions relative to neighbors get high LOF scores.

$$\text{LOF}(x) = \text{avg}(\text{lrd}(\text{neighbor})) / \text{lrd}(x)$$

$\text{LOF} \gg 1 \rightarrow$ anomaly (point is in sparser region than neighbors)

$\text{LOF} \approx 1 \rightarrow$ normal

Autoencoder-based:

Train an autoencoder on normal data. At inference, anomalies have high **reconstruction error** because the model never learned to reconstruct them.

Normal: Input $\rightarrow$ Encoder $\rightarrow$ Latent $\rightarrow$ Decoder $\rightarrow \approx$ Input (low error)

Anomaly: Input $\rightarrow$ Encoder $\rightarrow$ Latent $\rightarrow$ Decoder $\rightarrow \neq$ Input (high error)

Threshold reconstruction error to flag anomalies

24. Time Series & Forecasting

Theory

Time series data has a **temporal ordering** that makes standard ML assumptions invalid — observations are not independent. Specialized methods account for trends, seasonality, and autocorrelation.

Components of time series:

- **Trend:** Long-term increase or decrease
- **Seasonality:** Regular periodic fluctuations (daily, weekly, annual)
- **Cyclical:** Irregular fluctuations over longer periods
- **Noise:** Random variation

Decomposition:

Additive: $y(t) = \text{Trend} + \text{Seasonal} + \text{Residual}$

Multiplicative: $y(t) = \text{Trend} \times \text{Seasonal} \times \text{Residual}$

Stationarity

A time series is **stationary** if its statistical properties (mean, variance) don't change over time. Most classical models require stationarity.

Test: Augmented Dickey-Fuller (ADF) test

H₀: series has unit root (non-stationary)

Reject H₀ → stationary

To achieve stationarity:

Differencing: $y'(t) = y(t) - y(t-1)$

Log transform: $y'(t) = \log(y(t))$

ARIMA

ARIMA(p, d, q) — AutoRegressive Integrated Moving Average

AR(p): $y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \dots + \phi_p y_{t-p} + \varepsilon_t$

Predicts from p past values

MA(q): $y_t = c + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \dots + \theta_q \varepsilon_{t-q}$

Predicts from q past errors

I(d): d = order of differencing to make stationary

Combined ARIMA(p,d,q):

$(1 - \phi_1 B - \dots - \phi_p B^p)(1 - B)^d y_t = (1 + \theta_1 B + \dots + \theta_q B^q) \varepsilon_t$

SARIMA adds seasonal terms: SARIMA(p,d,q)(P,D,Q)[m]

Autocorrelation

ACF(k) = $\text{corr}(y_t, y_{t-k})$ → helps choose MA order q

PACF(k) = $\text{corr}(y_t, y_{t-k} \mid y_{t-1}, \dots, y_{t-k+1})$ → helps choose AR order p

ACF cuts off at lag q → MA(q) model

PACF cuts off at lag p → AR(p) model

Prophet (Facebook/Meta)

Decomposable model: $y(t) = g(t) + s(t) + h(t) + \varepsilon_t$

- $\boxed{g(t)}$ = trend (linear or logistic growth)
- $\boxed{s(t)}$ = seasonality (Fourier series)
- $\boxed{h(t)}$ = holiday effects

Strong at handling **missing data, outliers, multiple seasonalities**.

ML Approaches for Time Series

Feature Engineering for ML on Time Series:

Lag features: $x(t-1), x(t-2), \dots, x(t-p)$

Rolling stats: `rolling_mean(7), rolling_std(7)`

Date features: `hour, day_of_week, month, is_holiday`

Differences: $x(t) - x(t-1)$

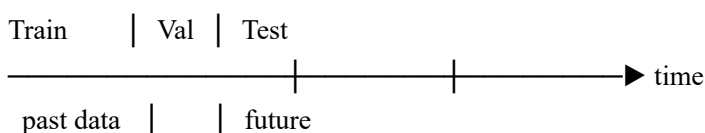
Models: XGBoost, LightGBM, Random Forest with these features
work surprisingly well for tabular time series

Deep Learning Approaches:

- **LSTM/GRU**: Sequential models, handles variable-length sequences
- **Temporal CNN (TCN)**: Dilated causal convolutions for long sequences
- **Temporal Fusion Transformer (TFT)**: State-of-the-art multi-horizon forecasting

Evaluation for Time Series

Always use time-aware splits:



NEVER randomly shuffle time series before splitting!

Walk-forward validation (expanding window):

Train: [1..100], Test: [101..110]

Train: [1..110], Test: [111..120]

...

25. Natural Language Processing (NLP)

Theory

NLP is the intersection of ML and linguistics — teaching computers to understand, interpret, and generate

human language. Language is inherently sequential, ambiguous, and context-dependent, making it one of the hardest domains.

Text Preprocessing Pipeline

Raw Text → Tokenize → Lowercase → Remove Stopwords
→ Stemming/Lemmatization → Vectorize → Model

Tokenization: Split text into tokens (words, subwords, characters)

Stemming vs Lemmatization:

Stemming (rule-based, faster):

"running" → "run"

"studies" → "studi" ← may not be real word

Lemmatization (dictionary-based, accurate):

"running" → "run"

"studies" → "study" ← actual base form

Text Vectorization

Bag of Words (BoW):

Vocabulary: ["cat", "dog", "sat", "mat"]

"cat sat" → [1, 0, 1, 0]

"dog sat" → [0, 1, 1, 0]

Loses word order. High-dimensional, sparse.

TF-IDF:

$TF(t, d) = \text{count}(t \text{ in } d) / \text{count}(\text{all words in } d)$

$IDF(t) = \log(N / df(t))$

N = total docs, $df(t)$ = docs containing t

$TF-IDF(t, d) = TF \times IDF$

High TF-IDF: word appears often in this doc but rarely across all docs

→ Captures discriminative importance of words

Word Embeddings (Word2Vec, GloVe):

Dense vector representations that capture semantic meaning. Similar words have similar vectors.

king - man + woman $\approx$ queen (famous vector arithmetic)

Word2Vec Training (Skip-gram):

Given center word → predict context words

Learn embeddings via backprop

Typical dim: 100–300

Cosine similarity:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \mathbf{a} \cdot \mathbf{b} / (\|\mathbf{a}\| \cdot \|\mathbf{b}\|)$$

Contextual Embeddings (BERT, etc.):

Unlike Word2Vec (static), BERT produces different embeddings for the same word based on context:

"bank" in "river bank" → different vector than

"bank" in "bank account"

Key NLP Tasks

TASK	MODEL/APPROACH
Text Classification	BERT + linear head, fastText
Named Entity Recog	BiLSTM-CRF, BERT-NER
Machine Translation	Seq2Seq + Attention, Transformer
Sentiment Analysis	Fine-tuned BERT, LSTM
Question Answering	BERT, RoBERTa (extractive)
Summarization	BART, T5, GPT (abstractive)
Text Generation	GPT family, LLaMA
Semantic Similarity	Siamese networks, Sentence-BERT

Fine-Tuning LLMs

Pre-trained LLM (massive general knowledge)

Fine-tuning on task-specific data



Specialized model

Approaches:

Full Fine-tuning: Update all parameters (expensive)

LoRA: Low-rank adapter matrices (efficient)

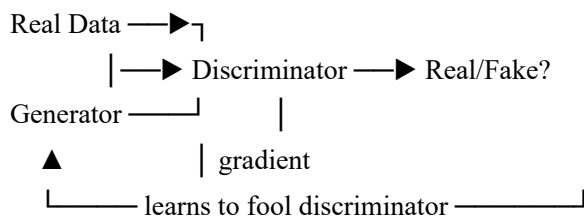
Prompt Tuning: Learn soft prompts (cheapest)

RLHF: Reinforce from human feedback (alignment)

26. Generative Models (GANs & VAEs)

26.1 Generative Adversarial Networks (GANs)

Theory: GANs consist of two networks in adversarial competition. The **Generator** creates fake samples; the **Discriminator** tries to distinguish real from fake. Through this competition, the Generator learns to produce increasingly realistic outputs.



Loss Functions:

Discriminator maximizes:

$$L_D = E[\log D(x)] + E[\log(1 - D(G(z)))]$$

↑ real correct ↑ fake correct

Generator minimizes:

$$L_G = E[\log(1 - D(G(z)))]$$

(or maximizes $E[\log D(G(z))]$ — non-saturating)

Nash equilibrium: G produces perfect fakes, D outputs 0.5 everywhere

Training challenges:

- **Mode collapse:** Generator produces only a few types of outputs
- **Training instability:** D and G need to be balanced
- **Vanishing gradient:** If D is too good early on, G can't learn

Variants:

DCGAN: Convolutional GAN for images

WGAN: Uses Wasserstein distance, more stable training

StyleGAN: Controls style at each layer; high-quality faces

CycleGAN: Unpaired image-to-image translation

Pix2Pix: Paired image-to-image translation

26.2 Variational Autoencoders (VAEs)

Theory: VAEs are generative models that learn a **continuous latent space** of data. Unlike regular autoencoders, the latent space is regularized to be smooth and continuous, enabling generation of new samples by sampling from the latent distribution.

Encoder: $x \rightarrow \mu, \sigma$ (learn distribution, not point)

Reparameterize: $z = \mu + \sigma \cdot \varepsilon, \varepsilon \sim N(0,1)$

Decoder: $z \rightarrow \hat{x}$

Latent space is Gaussian: $z \sim N(\mu, \sigma^2)$

ELBO Loss:

$L = \text{Reconstruction Loss} + \text{KL Divergence}$

Reconstruction: $E[\log p(x|z)]$

→ how well decoder reconstructs input

KL divergence: $KL(q(z|x) \parallel p(z))$

$= -0.5 \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)$

→ regularizes latent space to be $N(0,1)$

Latent Space Interpolation:

$z_1 \xrightarrow{\hspace{1.5cm}} z_2$

(image A) interpolate (image B)



Decode Decode Decode

(img A) (transition) (img B)

Diffusion Models (Modern Generative AI)

Theory: Diffusion models learn to **reverse a noise-adding process**. Forward process gradually adds Gaussian noise to data; the model learns the reverse denoising process.

Forward (fixed): $x_0 \rightarrow x_1 \rightarrow x_2 \rightarrow \dots \rightarrow x_t \sim N(0, I)$

(gradually add noise)

Reverse (learned): $x_t \rightarrow x_{t-1} \rightarrow \dots \rightarrow x_0$

(model predicts noise at each step)

Noise prediction:

$$L = E[||\epsilon - \epsilon\theta(x_t, t)||^2]$$

$\epsilon\theta$ = U-Net that predicts the noise added at step t

Examples: DALL-E 2, Stable Diffusion, Imagen, Midjourney

27. Model Interpretability (XAI)

Theory

Explainable AI (XAI) addresses the "black box" problem. As models grow more complex, understanding *why* they make predictions becomes critical — for trust, debugging, regulatory compliance, and detecting bias.

Local interpretability: Explain one specific prediction.

Global interpretability: Understand overall model behavior.

SHAP (SHapley Additive exPlanations)

Based on game theory's Shapley values. Each feature gets a credit for its contribution to the prediction, accounting for all possible feature orderings.

SHAP value for feature j :

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F|-|S|-1)!}{|F|!} \cdot [f(S \cup \{j\}) - f(S)]$$

Intuition: Average marginal contribution of feature j

across all possible feature subsets S

Properties:

Efficiency: $\sum \phi_j = f(x) - E[f(X)]$ (values sum to prediction)

Symmetry: Equal features get equal values

Dummy: Irrelevant features get $\phi=0$

SHAP Plot Types:

Force plot: —————[base]—[feat1+]—[feat2-]—► prediction

Shows push/pull of features for one prediction

Summary plot: Each feature × each sample, color = feature value

Shows global importance distribution

Waterfall: Step-by-step how each feature moves prediction

LIME (Local Interpretable Model-agnostic Explanations)

Approximates complex model locally with a simple interpretable model:

1. Pick a prediction to explain
2. Generate perturbed samples around input x
3. Get complex model's predictions for perturbed samples
4. Weight perturbed samples by proximity to x
5. Fit simple linear model on weighted samples
6. Use linear model coefficients as explanation

Feature Importance Methods

Permutation Importance:

1. Compute baseline score
2. Randomly shuffle feature j
3. Recompute score
4. Importance = drop in score

(model-agnostic, measures actual impact)

Built-in Tree Importance:

= average reduction in impurity from splits on feature j
(fast but biased toward high-cardinality features)

Partial Dependence Plots (PDP)

Shows **marginal effect** of one or two features on prediction:

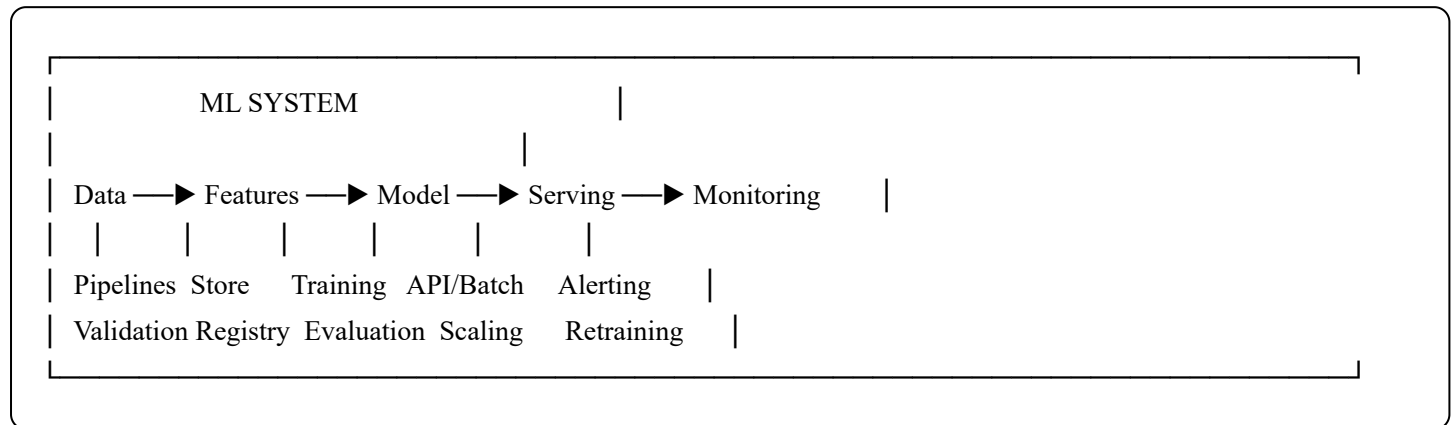
$$\text{PDP}(x_j) = E_{X_{-j}}[f(x_j, X_{-j})] = (1/n) \sum_i f(x_j, x_{-j}^{(i)})$$

Averaged over all other features — shows relationship
between feature j and target, marginalizing out other features

Theory

MLOps (Machine Learning Operations) applies DevOps principles to ML systems. The challenge: ML systems can fail silently — the model runs but produces wrong answers due to data drift, concept drift, or infrastructure changes.

A key insight from practitioners: **only a small fraction of ML code is the model itself**. The rest is data pipelines, feature stores, monitoring, serving infrastructure, and tooling.



ML Pipeline Stages

1. Data Validation

Check for:

- Schema changes (new/missing columns)
- Distribution shifts (mean, std, range)
- Missing value rates changing
- Data volume anomalies

Tools: Great Expectations, TFX Data Validation

2. Feature Store

Centralized repository of features shared across teams and models. Ensures consistent features between training and serving.

Offline store: For training (e.g., Hive, Parquet)

Online store: For low-latency serving (e.g., Redis, DynamoDB)

Point-in-time correctness: Feature values at time of event,
not current values (prevents label leakage)

3. Model Versioning & Registry

MLflow / Weights & Biases track:

- Model parameters and hyperparameters
- Training metrics at each epoch
- Artifacts (model files, plots)
- Dataset versions used

Registry stages: Staging → Production → Archived

4. Model Serving Patterns

Online Serving (REST API):

Request → Feature lookup → Model inference → Response

Latency: <100ms typically required

Batch Serving:

Scheduled job → Run predictions on dataset → Store results

For non-real-time use cases

Streaming:

Kafka topic → Feature computation → Model → Kafka output

For real-time event processing

5. Monitoring

What to monitor:

Data Drift: Input feature distributions shifting

PSI (Population Stability Index)

KS test, Wasserstein distance

Concept Drift: Relationship between X and y changing

Monitor model output distribution

Performance: Accuracy, latency, throughput

Error rates, null rate in predictions

Infrastructure: Memory, CPU, GPU utilization

6. Retraining Strategies

Scheduled: Retrain on fixed schedule (daily, weekly)

Triggered: Retrain when drift metric exceeds threshold

Online: Continuous learning from new data stream

Champion-Challenger: A/B test new model vs production model

Code Push → Unit Tests → Data Validation → Train on Subset
→ Evaluate → Compare to Baseline → Shadow Deploy
→ A/B Test → Full Rollout → Monitor

29. Imbalanced Data Techniques

Theory

Class imbalance occurs when one class significantly outnumbers another — common in fraud detection (0.1% fraud), medical diagnosis (rare diseases), or churn prediction. Standard models trained on imbalanced data tend to predict the majority class almost always, achieving high accuracy while completely failing on the minority class.

Example: 99% negative, 1% positive. A model that always predicts "negative" gets 99% accuracy but 0% recall on positives.

Resampling Methods

OVERSAMPLING	UNDERSAMPLING
Duplicate minority samples (or generate synthetic ones)	Remove majority samples
SMOTE: 1. Pick minority pt 2. Find k neighbors 3. Generate point on the line between them	Random Undersampling Tomek Links: Remove majority pts that are near minority pts Cluster Centroids: Replace cluster with its centroid

SMOTE (Synthetic Minority Over-sampling Technique):

- For each minority sample x_i :
- Find k nearest minority neighbors
 - Pick random neighbor x_n
 - Generate: $x_{\text{new}} = x_i + \lambda(x_n - x_i)$, $\lambda \sim \text{Uniform}(0,1)$

Algorithm-Level Methods

Class Weights:

```
weight_positive = n_total / (2 · n_positive)  
weight_negative = n_total / (2 · n_negative)
```

```
Loss = -[w+ · y · log( $\hat{y}$ ) + w- · (1-y) · log(1- $\hat{y}$ )]
```

```
sklearn: class_weight='balanced'
```

Focal Loss (used in object detection):

$$FL(p) = -\alpha(1-p_i)^{\gamma} \cdot \log(p_i)$$

$(1-p_i)^{\gamma}$: down-weights easy examples ($\gamma > 0$)

α : class weight

Easy correct predictions contribute less to loss

→ model focuses on hard minority examples

Threshold Adjustment:

Default threshold = 0.5

For imbalanced data: lower threshold to increase recall

Find optimal threshold on validation set using:

- F1 score curve
- Precision-Recall curve
- Business cost matrix

Evaluation Under Imbalance

Avoid accuracy! Use instead:

- Precision-Recall AUC (PR-AUC) ← best for imbalanced
 - F1 Score (or F_{β} score)
 - Cohen's Kappa
 - Matthews Correlation Coefficient (MCC)
$$= (TP \cdot TN - FP \cdot FN) / \sqrt{((TP+FP)(TP+FN)(TN+FP)(TN+FN))}$$
- Range: [-1, 1], 1 = perfect, 0 = random

30. Advanced Optimization Algorithms

Theory

Optimization is at the heart of ML training. Beyond vanilla gradient descent, modern optimizers adapt learning rates per-parameter, use momentum to navigate loss landscapes better, and handle noisy gradients efficiently.

Optimizer Comparison

SGD:

$$\theta := \theta - \alpha \nabla L$$

Simple but requires careful LR tuning; no adaptivity

Momentum:

$$v := \beta v + \nabla L \quad \beta \approx 0.9$$

$$\theta := \theta - \alpha v$$

Accelerates in consistent gradient directions

Dampens oscillations

Nesterov Accelerated Gradient (NAG):

$$v := \beta v + \nabla L(\theta - \beta v) \quad \leftarrow \text{gradient at "lookahead" point}$$

$$\theta := \theta - \alpha v$$

Corrects momentum before applying it

AdaGrad:

$$G := G + (\nabla L)^2$$

$$\theta := \theta - \alpha / \sqrt{G + \epsilon} \cdot \nabla L$$

Per-parameter LR; large updates for rare features

Problem: LR shrinks monotonically $\rightarrow$ training stalls

RMSProp:

$$G := \beta G + (1 - \beta)(\nabla L)^2 \quad \leftarrow \text{exponential moving average}$$

$$\theta := \theta - \alpha / \sqrt{G + \epsilon} \cdot \nabla L$$

Fixes AdaGrad's shrinking LR problem

Adam (Adaptive Moment Estimation):

$$m := \beta_1 m + (1 - \beta_1) \nabla L \quad \leftarrow \text{1st moment (mean)}$$

$$v := \beta_2 v + (1 - \beta_2) (\nabla L)^2 \quad \leftarrow \text{2nd moment (variance)}$$

$$\hat{m} := m / (1 - \beta_1^t) \quad \leftarrow \text{bias correction}$$

$$\hat{v} := v / (1 - \beta_2^t)$$

$$\theta := \theta - \alpha \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$$

Defaults: $\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-8$, $\alpha=0.001$

Most widely used optimizer in deep learning

AdamW:

Adam + decoupled weight decay

$$\theta := \theta - \alpha \cdot [\hat{m}/(\sqrt{\hat{v}} + \varepsilon) + \lambda\theta]$$

Preferred for Transformer training (BERT, GPT use AdamW)

Learning Rate Schedules

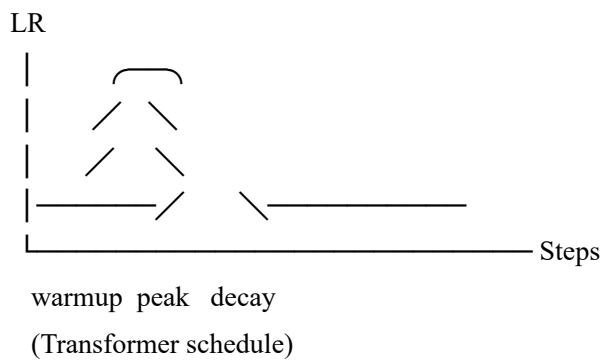
Constant: $\alpha = 0.001$ (simplest)

Step Decay: $\alpha := \alpha \cdot \text{drop_rate}$ every k epochs
e.g., halve every 10 epochs

Cosine Annealing: $\alpha(t) = \alpha_{\min} + 0.5(\alpha_{\max} - \alpha_{\min})(1 + \cos(\pi t/T))$
Smooth decay, widely used

Warm-up + Decay: LR increases linearly for W steps,
then decays (standard for Transformers)

One-Cycle Policy: LR increases to max then decreases
(fast training, used in fastai)



Gradient Clipping

Prevents **exploding gradients** (common in RNNs and deep networks):

By value: $g := \text{clip}(g, -c, c)$

By norm: if $\|g\| > c$: $g := c \cdot g/\|g\|$

Typical $c = 1.0$ or 5.0

31. Probability Distributions in ML

Theory

Many ML algorithms are grounded in probabilistic assumptions about data. Understanding which distribution to use for modeling different types of data is fundamental to building correct models and interpreting outputs.

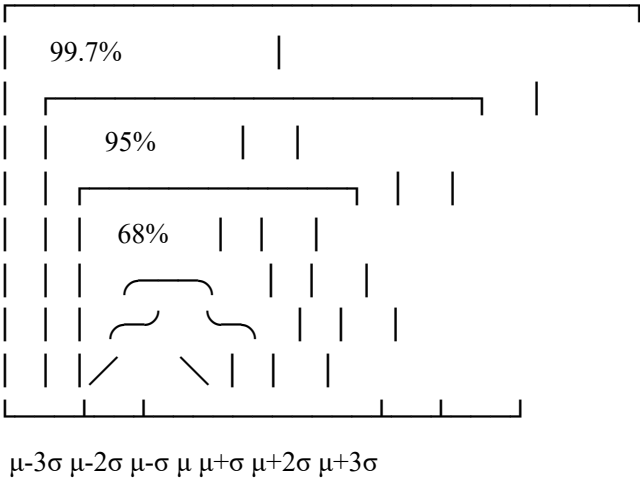
Key Distributions

DISTRIBUTION	SUPPORT	PARAMETERS	USE IN ML
Bernoulli	{0,1}	p	Binary classification
Binomial	0..n	n,p	Count successes
Categorical	1..K	p1..pK	Multiclass output
Gaussian	$\mathbb{R}$	μ,σ	Regression, noise model
Poisson	0,1,2..	λ	Count events
Exponential	$\mathbb{R}_{\geq 0}$	λ	Time-to-event, survival
Beta	[0,1]	α,β	Probability estimation
Dirichlet	simplex	$\alpha_1..\alpha_K$	Topic modeling (LDA)
Laplace	$\mathbb{R}$	μ,b	Robust regression (L1)

Gaussian Distribution in Depth

$f(x) = (1/\sigma\sqrt{2\pi}) \exp(-(x-\mu)^2/2\sigma^2)$

- 68-95-99.7 Rule:
- 68% of data within 1σ
 - 95% of data within 2σ
 - 99.7% of data within 3σ



Maximum Likelihood Estimation (MLE)

The principle underlying most ML model training:

Find parameters θ that maximize the likelihood of observed data:

$$\begin{aligned}\theta_{\text{MLE}} &= \operatorname{argmax}_{\theta} L(\theta) = \operatorname{argmax}_{\theta} \prod P(x_i|\theta) \\ &= \operatorname{argmax}_{\theta} \sum \log P(x_i|\theta) \quad (\text{log-likelihood, easier})\end{aligned}$$

For Gaussian: MLE gives $\theta = (\mu=\bar{x}, \sigma^2=\text{sample variance})$

For Bernoulli: MLE gives $p = \text{fraction of 1s}$

MAP vs MLE

MLE: $\theta_{\text{MLE}} = \operatorname{argmax}_{\theta} P(\text{data}|\theta)$

MAP: $\theta_{\text{MAP}} = \operatorname{argmax}_{\theta} P(\text{data}|\theta) \cdot P(\theta)$
 $= \operatorname{argmax}_{\theta} [\log P(\text{data}|\theta) + \log P(\theta)]$

MAP adds a prior term — this is equivalent to regularization!

Gaussian prior on $\theta \rightarrow$ L2 regularization (Ridge)

Laplace prior on $\theta \rightarrow$ L1 regularization (Lasso)

32. Graph Neural Networks

Theory

Graph Neural Networks (GNNs) extend deep learning to **graph-structured data**, where relationships between entities are first-class citizens. Unlike images (regular grid) or text (linear sequence), graphs have irregular, variable-size neighborhoods.

Applications: Social networks, molecular property prediction, recommendation systems, knowledge graphs, fraud detection, traffic forecasting.

Graph Basics

Graph $G = (V, E)$

V = set of nodes (vertices)

E = set of edges

Node features: $X \in \mathbb{R}^{(|V| \times d)}$ (d features per node)

Adjacency matrix: $A \in \{0,1\}^{(|V| \times |V|)}$

Types:

Undirected / Directed

Weighted / Unweighted

Homogeneous / Heterogeneous

Message Passing Framework

The core idea: **each node aggregates information from its neighbors** to update its own representation. After k layers, each node has information from its k -hop neighborhood.

Step 1 — Message:

$$m_{j \rightarrow i} = M(h_i, h_j, e_{ij}) \quad \text{for each neighbor } j \text{ of } i$$

Step 2 — Aggregate:

$$a_i = \text{AGG}(\{m_{j \rightarrow i} : j \in N(i)\})$$

(Sum, Mean, Max, or learned aggregation)

Step 3 — Update:

$$h_i' = \text{UPDATE}(h_i, a_i)$$

(typically a neural network)

Graph Convolutional Network (GCN)

$$H^{(l+1)} = \sigma(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)})$$

Where:

$$\tilde{A} = A + I \quad (\text{adjacency} + \text{self-loops})$$

$\tilde{D}$ = degree matrix of $\tilde{A}$

$W^{(l)}$ = learnable weight matrix for layer l

σ = activation function

Simplified: each node = average of its neighbors' features
passed through a linear layer

GNN Variants

GCN: Spectral convolution; simple mean aggregation

GraphSAGE: Sample and aggregate; inductive (generalizes to new nodes)

GAT: Graph Attention Networks; learns attention weights over neighbors

GIN: Graph Isomorphism Network; maximally expressive for graph classification

MPNN: General message passing neural network framework

33. Self-Supervised & Contrastive Learning

Theory

Self-supervised learning creates **supervisory signals from the data itself**, without human labels. This allows leveraging massive unlabeled datasets for pre-training, followed by fine-tuning on small labeled datasets. It's now the dominant paradigm for foundation models.

Key idea: Define a pretext task where the labels come for free from the data structure (e.g., predict missing words, predict next frame, predict which patches belong together).

Contrastive Learning

Learn representations such that **similar (positive) pairs are close** in embedding space and **dissimilar (negative) pairs are far apart**.

Positive pair: (augmented view 1 of image X, augmented view 2 of image X)

Negative pair: (image X, image Y)

Loss pulls positives together, pushes negatives apart

NT-Xent Loss (SimCLR):

$$L(i,j) = -\log [\exp(\text{sim}(z_i, z_j)/\tau) / \sum_{k \neq i} \exp(\text{sim}(z_i, z_k)/\tau)]$$

$$\text{sim}(u,v) = u^T v / (\|u\| \cdot \|v\|) \quad (\text{cosine similarity})$$

τ = temperature hyperparameter

Data Augmentation for Images:

Random crop → Random horizontal flip → Color jitter

→ Grayscale → Gaussian blur → Two views of same image

BERT's Pre-training Tasks

1. Masked Language Modeling (MLM):

Input: "The [MASK] sat on the mat"

Target: "cat"

→ Predict 15% of randomly masked tokens

2. Next Sentence Prediction (NSP):

Input: Sentence A + [SEP] + Sentence B

Target: IsNext / NotNext

→ Predict if B follows A

SimCLR, MoCo, BYOL Progression

SimCLR: Large batch for negatives, two-view, end-to-end

MoCo: Momentum encoder + memory bank (memory-efficient)

BYOL: No negatives! Online → Target network (with momentum)

Avoids collapse via asymmetry + batch normalization

SimSiam: No negatives, no momentum, stop-gradient prevents collapse

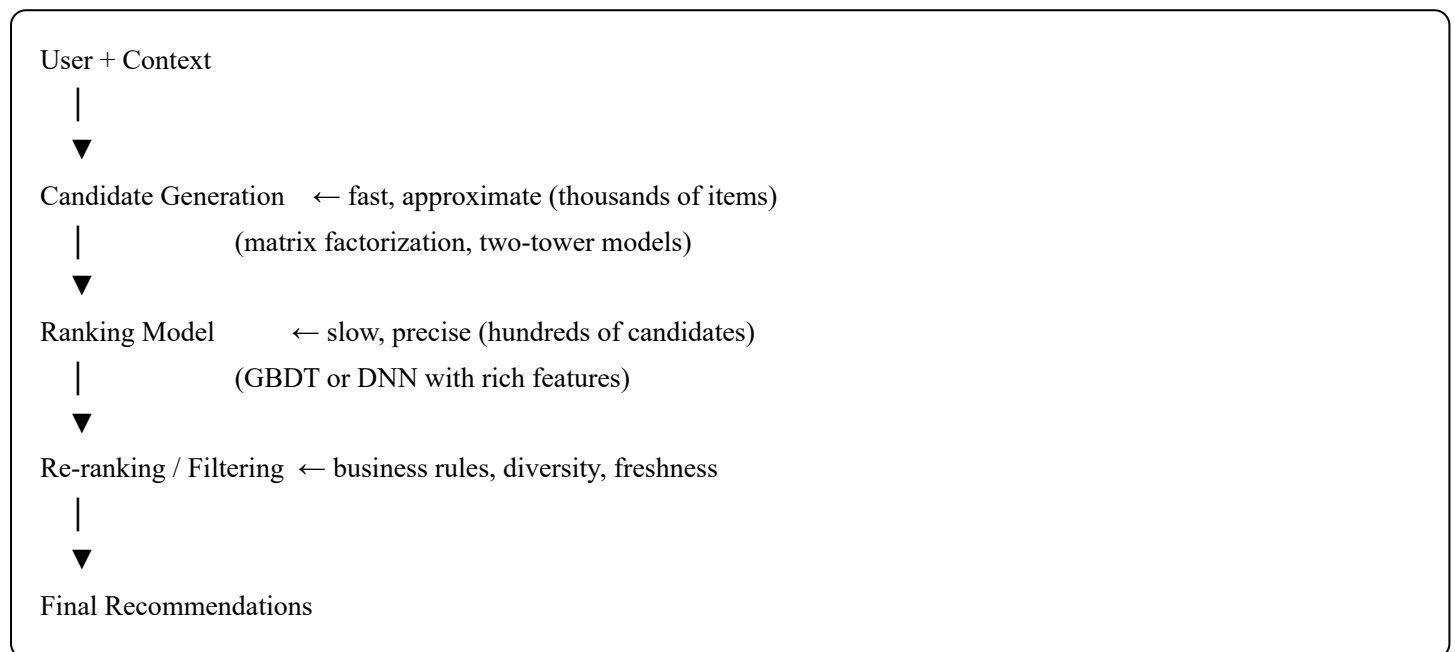
34. ML System Design Patterns

Theory

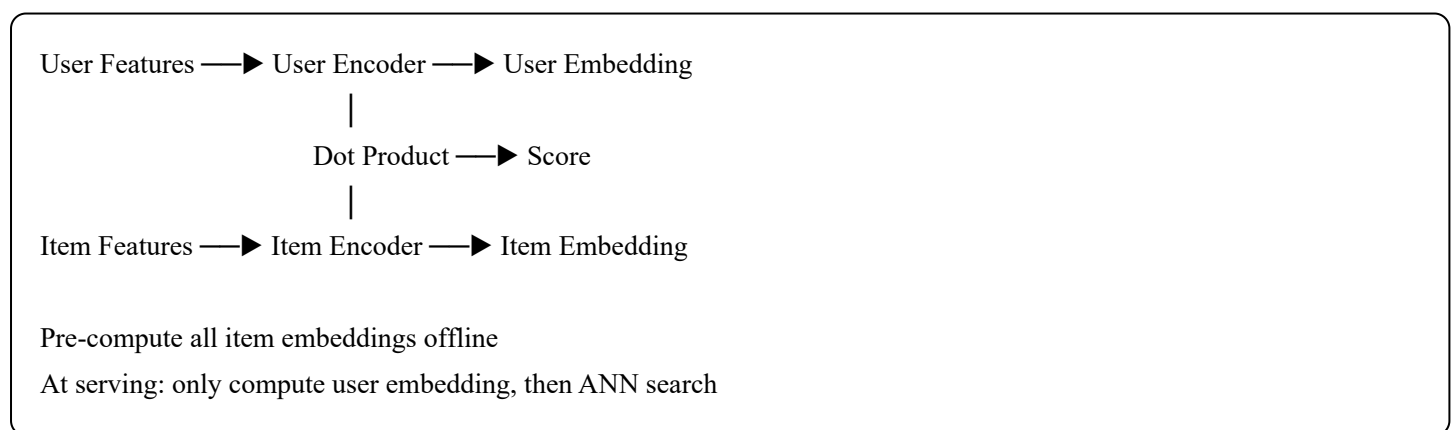
Designing ML systems for scale requires thinking beyond model accuracy. You must consider data freshness, latency, throughput, consistency, and failure modes. Successful ML systems combine good models with robust engineering.

Common ML System Architectures

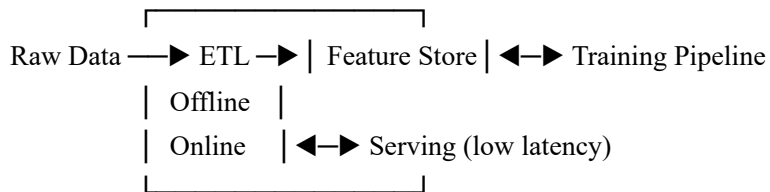
Recommendation System:



Two-Tower Model:



Feature Store Pattern:



Point-in-time join: critical to prevent train/serve skew

Train-Serve Skew

A critical production failure mode: features computed differently at training time vs serving time.

Training: $\text{age} = (\text{current_date} - \text{birth_date})$ using batch date

Serving: $\text{age} = (\text{request_timestamp} - \text{birth_date})$ using live timestamp

Small mismatch $\rightarrow$ large silent performance degradation

Fix: Use feature store with consistent computation logic

Data Splits for Different Scenarios

Standard:

Random 70/15/15 $\rightarrow$ OK for i.i.d. tabular data

Time-Based:

[train][val][test] $\rightarrow$ Required for time series

Group-Based:

Users A-Z $\rightarrow$ split by user_id $\rightarrow$ prevents data leakage

(if user can appear in both train and test, model memorizes user)

Spatial:

Split by geographic region $\rightarrow$ tests generalization to new areas

35. Python ML Ecosystem Cheatsheet

Core Libraries Quick Reference

python

Data Manipulation

```
import numpy as np      # Arrays, linear algebra
import pandas as pd     # DataFrames, data wrangling
```

Visualization

```
import matplotlib.pyplot as plt # Plots
import seaborn as sns          # Statistical visualization
import plotly.express as px    # Interactive plots
```

Classical ML

```
from sklearn.linear_model import LinearRegression, LogisticRegression, Ridge, Lasso
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC, SVR
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB, MultinomialNB
from sklearn.cluster import KMeans, DBSCAN, AgglomerativeClustering
from sklearn.decomposition import PCA, TruncatedSVD
from sklearn.preprocessing import StandardScaler, MinMaxScaler, LabelEncoder, OneHotEncoder
from sklearn.model_selection import train_test_split, KFold, GridSearchCV, RandomizedSearchCV
from sklearn.metrics import (accuracy_score, precision_score, recall_score, f1_score,
                             roc_auc_score, mean_squared_error, r2_score, confusion_matrix)
from sklearn.pipeline import Pipeline
```

Gradient Boosting

```
import xgboost as xgb
import lightgbm as lgb
import catboost as cb
```

Deep Learning

```
import torch
import torch.nn as nn
import tensorflow as tf
from tensorflow import keras
```

NLP

```
import nltk
from transformers import AutoTokenizer, AutoModel, pipeline # HuggingFace
import spacy
```

MLOps

```
import mlflow
import wandb
```

Sklearn Pipeline Pattern

```
python

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import RandomForestClassifier

# Build pipeline: preprocessing + model
pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', RandomForestClassifier(n_estimators=100))
])

# Train
pipe.fit(X_train, y_train)

# Predict (automatically applies scaler + model)
y_pred = pipe.predict(X_test)

# Cross-validate
from sklearn.model_selection import cross_val_score
scores = cross_val_score(pipe, X, y, cv=5, scoring='f1')
```

Training Loop in PyTorch

```
python
```



```

model = MyModel()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
criterion = nn.CrossEntropyLoss()

for epoch in range(num_epochs):
    model.train()
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()      # Clear gradients
        y_pred = model(X_batch)     # Forward pass
        loss = criterion(y_pred, y_batch) # Compute loss
        loss.backward()             # Backpropagation
        optimizer.step()            # Update weights

    # Validation
    model.eval()
    with torch.no_grad():
        val_preds = model(X_val)
        val_loss = criterion(val_preds, y_val)
    print(f"Epoch {epoch}: train={loss:.4f}, val={val_loss:.4f}")

```

Keras Quick Model

```

python

model = keras.Sequential([
    keras.layers.Dense(128, activation='relu', input_shape=(n_features,)),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(64, activation='relu'),
    keras.layers.Dropout(0.3),
    keras.layers.Dense(1, activation='sigmoid') # binary output
])

model.compile(
    optimizer='adam',
    loss='binary_crossentropy',
    metrics=['accuracy', keras.metrics.AUC()]
)

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=50,
    batch_size=32,
    callbacks=[keras.callbacks.EarlyStopping(patience=5, restore_best_weights=True)]
)

```

XGBoost Quick Reference

```
python

import xgboost as xgb

model = xgb.XGBClassifier(
    n_estimators=500,
    learning_rate=0.05,
    max_depth=6,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_alpha=0.1,    # L1
    reg_lambda=1.0,   # L2
    use_label_encoder=False,
    eval_metric='logloss',
    early_stopping_rounds=20
)

model.fit(X_train, y_train,
          eval_set=[(X_val, y_val)],
          verbose=50)

# Feature importance
xgb.plot_importance(model)
```

SHAP in 5 Lines

```
python

import shap

explainer = shap.TreeExplainer(model)    # or shap.Explainer(model)
shap_values = explainer.shap_values(X_test)

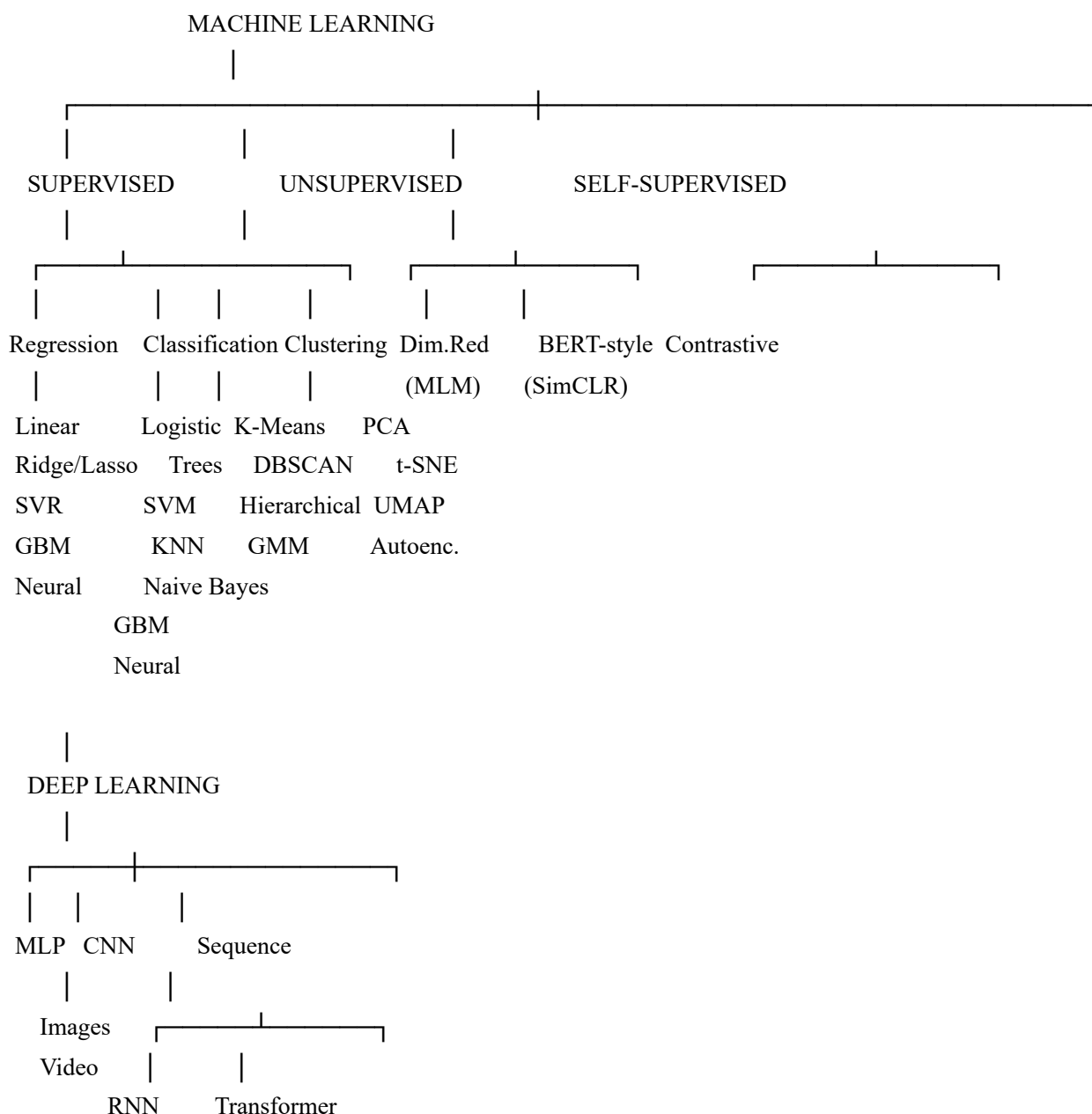
shap.summary_plot(shap_values, X_test)    # Global importance
shap.force_plot(explainer.expected_value, # Single prediction
                 shap_values[0], X_test.iloc[0])
```

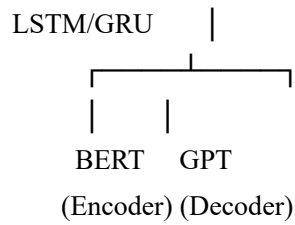
Common Gotchas & Tips

- ⚠ Always split BEFORE any fitting (scaler, imputer, encoder)
→ Prevents data leakage from validation into training
- ⚠ Time series: NEVER random split, always time-based

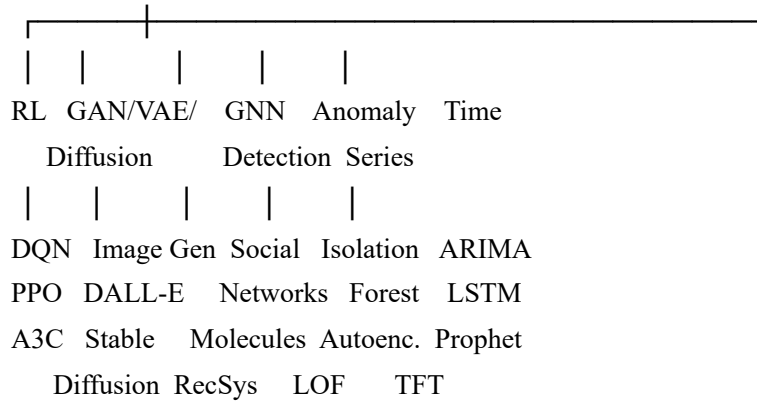
- ⚠ One-hot encoding: `drop_first=True` to avoid multicollinearity
- ⚠ Tree models: don't need scaling; neural nets do
- ⚠ GridSearchCV: use `refit=True` (default) to get best model
- ⚠ Class imbalance: set `class_weight='balanced'` or use SMOTE
- ✓ Use pipelines to prevent leakage and simplify deployment
- ✓ Log experiments (MLflow/wandb) even for small projects
- ✓ Set `random_state` everywhere for reproducibility
- ✓ Profile data with pandas-profiling or ydata-profiling first
- ✓ Check feature importances early to guide feature engineering

📖 Extended Concept Map



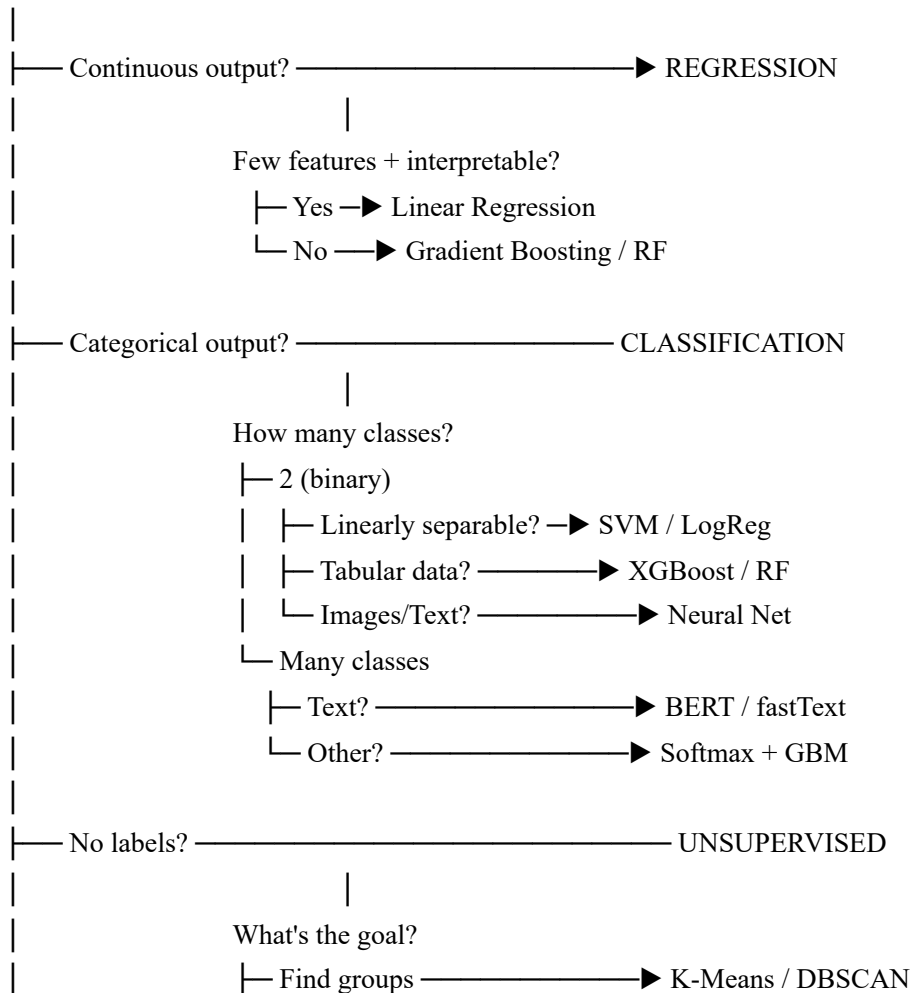


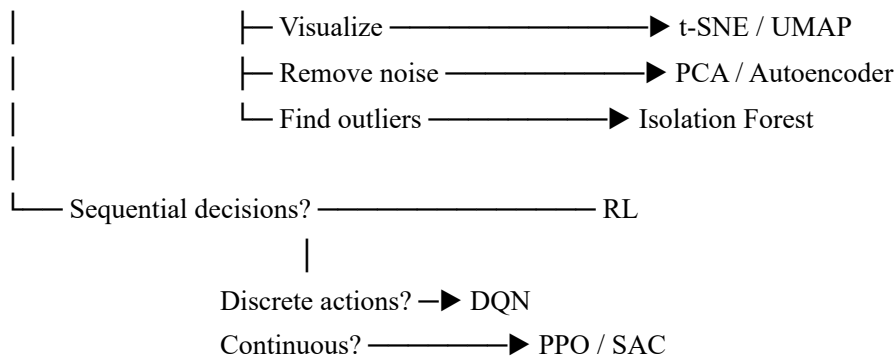
SPECIAL TOPICS



Decision Flowchart: Choosing Your Algorithm

START: What is your problem?





🚀 Updated Golden Rules (Extended)

1. **Start simple.** A logistic regression or linear model is your baseline — always.
2. **Data > Algorithm.** More quality data beats a fancy model on bad data.
3. **Validate properly.** Never evaluate on training data; use held-out test set.
4. **Scale your features.** Most algorithms except trees need scaled inputs.
5. **Understand your metric.** Accuracy is misleading on imbalanced data.
6. **Regularize.** When in doubt, add L2 regularization.
7. **Ensembles win.** Combining models almost always beats a single model.
8. **Feature engineering matters.** Transforming features intelligently often beats hyperparameter tuning.
9. **Monitor in production.** Models degrade as data distribution shifts.
10. **Iterate.** Build fast, evaluate honestly, improve incrementally.
11. **Prevent leakage.** Always fit preprocessing on training data only.
12. **Explain your model.** Use SHAP/LIME to build trust and catch bugs.
13. **Version everything.** Data, code, and models should all be versioned.
14. **Beware of imbalance.** Check class distributions before evaluating.
15. **Respect temporal structure.** Time series data must be split chronologically.
16. **Think about the cost of errors.** Tune thresholds based on business consequences, not just F1.
17. **Self-supervised pre-training.** When labels are scarce, leverage unlabeled data.
18. **Attention to attention.** Transformer-based models are the default for text and increasingly for other modalities.
19. **GNNs for relational data.** When data has natural graph structure, use it.
20. **MLOps is not optional.** A model that can't be deployed or monitored has no production value.